

Mixed Arithmetic-Binary Circuits in Fluid MPC Against Honest Majority of 4-Party and Its Applications Against Semi-Honest Adversary

Furkan Kerim Çabaş^{1*} and Oğuz Yayla¹

¹Cryptography, Middle East Technical University, Ankara, 06800,
Türkiye.

*Corresponding author(s). E-mail(s): kcabas@metu.edu.tr;
Contributing authors: oguz@metu.edu.tr;

Abstract

Secure multi party computation protocols (MPC) translating between arithmetic and binary data types have recently gained attraction which is introduced by Rotaru and Wood in 2019, called daBit, and improved by Escudero et. al. called edaBits. EdaBits are simply secret shares in arithmetic domain and bit decomposition of the arithmetic share is the binary form the secret shares. These protocols are preprocessing for MPC protocols in order to improve efficiency. Furthermore, fluid MPC setting, introduced by Choudhuri, Goel, Green, Jain and Kaptchuk in 2021, is a framework which the parties, called dynamic parties, do not have to be online throughout the whole process. Fluid MPC framework is still has to be worked and conventional protocols and techniques has to be adapted for it. Our work introduces an edaBits protocol specifically designed for the fluid MPC setting under a four-party honest-majority model. The protocol, which consists of eight subprotocols, achieves security against an honest majority and departs slightly from the traditional cut-and-choose paradigm. It attains linear memory and time complexity, as well as constant communication complexity. Finally, we demonstrate two applications that employ edaBits within the fluid MPC framework under the semi-honest adversary model.

Keywords: Secure Multi-Party Computation (MPC), Fluid MPC, edaBits, Privacy-Preserving Machine Learning

1 Introduction

In cryptography, one of the most important purpose is to send the information to other entities without leaking any information about the information. Simple methods such as substitution method was used in early times of cryptography in order to achieve the goal. As time and technology progresses, mathematics is involved into cryptography and symmetric and asymmetric cryptographic methods have invented. These methods are between two parties and one entity has an information and the other entity wants this information secretly. However, if more than two entities wants to share their secrets to obtain a result, we did not have a good method until the beginning of 1980s. Yao in the early 1980s introduced the well-known *Millionaire Problem* [1]. The problem is formulated as follows: two millionaires are having lunch and agree that the one with greater wealth will pay the bill, but neither wishes to reveal their actual wealth to the other. Here, the wealth of each millionaire is considered their private input, and the individual who pays the bill corresponds to the output of the function. The function itself is securely evaluated using cryptographic techniques. This idea laid the foundation of a cryptographic technique which more than two parties can jointly evaluate a function and obtain a result and this set of techniques were called as *Secure Multi-Party Computation*

Secure Multi-Party Computation (MPC) is a cryptographic set of techniques that enables multiple parties to jointly compute a function f over their private inputs, such that no party learns anything beyond the final output. The concept was initially introduced by Yao in the early 1980s in his influential paper [1], and has since inspired the development of several MPC protocols. For a comprehensive and foundational overview of MPC, the reader is referred to [2–6].

An essential aspect of MPC is the consideration of adversarial behavior. In general, two main types of adversaries are distinguished: *passive* and *active*. Passive adversaries follow to the designated protocol execution but attempt to gain of additional information. In contrast, active adversaries may deviate from the protocol either to gain additional information or to disrupt the computation. MPC protocols are further classified based on their endurance to number of corrupted parties and this structure is called *threshold structure*. Threshold structure measures the power of the adversary by how many parties the adversary can corrupt. A protocol is said to be secure in the *honest-majority* setting if fewer than half of the parties are corrupted. In contrast, it is said to tolerate a *dishonest majority* if security is preserved even when all but one party may be corrupted.

MPC protocols typically operate in either a binary or arithmetic domain. In the early development of MPC, it was often sufficient for a protocol to operate entirely within one domain. However, as applications have developed more complex, it has become necessary to support computations that include both binary and arithmetic operations. For instance, privacy-preserving machine learning protocols may use both binary and arithmetic algorithms in order to improve the efficiency of the protocols. Consequently, efficient conversion between binary and arithmetic shares is important. Several works have addressed this problem, including [7–11]. In particular, Rotaru and Wood [11] introduced the notion of *doubly authenticated Bits*, which was subsequently extended by Escudero *et al.* [8] called *edaBits*.

In short, an edaBit is a tuple consisting of secret shares in both the arithmetic and binary domains that represent the same underlying value. This value is generated by aggregating secret arithmetic and binary shares that are jointly sampled and distributed by all participating parties. edaBits serves as a cryptographic primitive used during the preprocessing phase to enhance the efficiency of secure computation protocols, and were originally introduced for use in conventional MPC settings. However, the generation of edaBits introduces several challenges. Since edaBits are collaboratively constructed by multiple parties, a central difficulty lies in ensuring consistency between their arithmetic and binary components, particularly in the presence of an active adversary.

Our contribution lies in the development of edaBits within the fluid MPC setting. The central component of this work is the protocol $\Pi_{\text{private-edaBits}}$, which ensures consistency between the binary and arithmetic components of private edaBits in the fluid MPC environment. The goal is to enable the processing of both components independently and efficiently among parties. While arithmetic operations in the fluid MPC setting have been extensively studied and are relatively well-supported, binary operations have received less attention. As a result, additional techniques are required to support binary components in a manner similar to arithmetic operations. To address this issue, we introduce a novel communication mechanism specifically designed for handling the binary component of edaBits in the fluid MPC context. The protocol $\Pi_{\text{private-edaBits}}$ is designed under the assumption of an honest majority in a 4-party setting, enabling secure and consistent processing of the binary component of edaBits. Additionally, we demonstrate the application of these procedures in protocols that convert between binary and arithmetic representations, as discussed in the final section.

The remainder of this paper is organized as follows: In Section 2, we present the necessary preliminaries, including definitions of daBits and edaBits, and introduce the fluid MPC model. In Section 3, we introduce some essential functionalities and general design underlying our constructions. Section 4 introduces the $\Pi_{\text{edaBitsFluid}}$ protocol. In Section 5, we elaborate on our main contribution, the $\Pi_{\text{private-edaBits}}$ protocol, and describe how bit shares are transferred to subsequent committees. In Section 6, two applications of our approach: Arithmetic-to-Binary and Binary-to-Arithmetic conversion protocols are illustrated. Finally, Section 7 provides concluding remarks and discusses potential directions for future research.

2 Preliminaries

In 2019, Rotaru and Wood introduced the concept of *daBits* [11]. Subsequently, Escudero *et al.* [8] proposed *edaBits* as an extension of daBits. We now provide some intuition behind the constructions of daBits and edaBits. In a daBits construction, we have a pair consisting of an arithmetic share and a binary share of the same bit value, denoted as $([b]_M, [b]_2)$, where $b \in \{0, 1\}$, and the subscripts M and 2 refer to the modulus used for arithmetic and binary sharing, respectively. An edaBits is a daBits of length k . More concretely, an edaBits consists of a value $r \in \mathbb{Z}_M$ and its binary representation $(b_0, b_1, \dots, b_{k-1})$ with $b_i \in \mathbb{Z}_2$, where both the integer r and the bits b_i are secret

shared among the parties. The edaBit is represented as $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$ with the invariant relation $r = \sum_{i=0}^{k-1} b_i \cdot 2^i$. This ensures consistency between the arithmetic and binary representations of the same underlying value.

We now provide an intuition for how binary and arithmetic secret shares are generated and obtained in the context of edaBits. In the construction, we distinguish between two types of edaBits: *private* edaBits and *global* edaBits. In brief, private edaBits are the output of the protocol $\Pi_{\text{private-edaBits}}$ and serve as the input to the $\Pi_{\text{edaBitsFluid}}$ protocol. Global edaBits, on the other hand, are the output of $\Pi_{\text{edaBitsFluid}}$ and are suitable for use in higher-level applications.

The core idea behind the protocol is that each party generates its own private edaBits such that party P_i uniformly samples n bit shares corresponding to the first bit as $b_0 = b_{1,0} \oplus b_{2,0} \oplus \dots \oplus b_{n,0}$ and distributes each share $b_{j,0}$ to party P_j . Additionally, each party receives private edaBits from the other parties. Specifically, party P_j receives one bit from each of the n parties for the first bit position, forming the tuple $b_{j,0}, r_{j,0}, \dots, t_{j,0} \in \mathbb{Z}_2$, where $b_{j,0}, r_{j,0}, \dots, t_{j,0}$ denotes the first bit of the private edaBits obtained from different parties. Party P_j then locally aggregates these bits by computing $[b_0]_2 = b_{j,0} \oplus r_{j,0} \oplus \dots \oplus t_{j,0}$, where $[b_0]_2$ denotes the first bit of the global edaBits.

In this process, each party prepares its private edaBits as additive shares and distributes them among the other parties. This collective approach ensures that all parties contribute to a global edaBits. Consequently, the final result is a collection of global edaBits, where each party holds a share of a batched edaBits.

As a result, the parties collectively hold globally consistent edaBits, ensuring agreement between their binary and arithmetic representations. These global edaBits can then be used as preprocessing material in subsequent MPC protocols. The generation of private edaBits is carried out through the ideal functionality $\mathcal{F}_{\text{private-edaBits}}$, which will be described in detail in the following sections.

There exist several frameworks for formalizing the security proofs of MPC protocols, including [12–14]. These frameworks are mostly similar, with only minor differences in structure and notation. In this work, we adopt the *Universal Composability (UC) Framework* introduced by Canetti in [13]. The core concept of the UC Framework is the distinction between two conceptual worlds: the *real world* and the *ideal world*. In the ideal world, a *simulator* is used to simulate the behavior of the entire protocol. The security of a protocol is defined in terms of the indistinguishability between the real and ideal worlds specifically, the difference in the probability that an adversary can gain more information than allowed. Informally, if a protocol does not leak any additional information in the ideal world, then it is considered secure for use in the real world.

Several components are essential in the ideal world: one is the *environment* $\mathcal{Z}$, which governs the flow of information and interacts with the parties and the adversary. Another is the concept of a *functionality*, which can be thought of as a trusted third party. Functionality instances are connected to both the simulator and the environment.

A major advantage of the UC Framework is its composability. Complex protocols can be partitioned into subprotocols, each proven secure separately. Suppose a protocol

consists of two main components, and proving the security of the entire protocol from the beginning is challenging. The UC Framework allows one to first prove the security of a subprotocol and then treat it as an ideal functionality in the security proof of the larger protocol. According to the UC composition theorem, if each subprotocol is proven secure, then the composition of these protocols also remains secure. This composability leads to an important observation: a functionality $\mathcal{F}$ can be used in a protocol Π even if its implementation $\pi_{\mathcal{F}}$ is not yet defined. Once $\pi_{\mathcal{F}}$ is implemented, the overall security of Π is ensured due to the composition theorem.

Fluid MPC is a framework introduced by Choudhuri *et al.* [15] that brings more flexibility to traditional MPC protocols. In this setting, *fluidity* means that parties do not need to stay active or participate throughout the entire execution of a protocol. This is useful because MPC protocols can often take a long time to complete, and it may not be practical for all parties to remain online the whole time. Sometimes, parties might lose connection or quit, which can cause the whole protocol to fail even if the failure happens near the end. To address this problem, the fluid MPC model breaks the protocol into smaller steps. Each step can be handled by different parties, as long as enough of them are available at each stage. This approach helps avoid failures due to temporary or permanent disconnections, and makes the system more reliable in real world situations. After the introduction of fluid MPC in [15], several other works have continued to explore and improve the model, including [16–18].

Fluid MPC follows a client/server model, where a client provides secret inputs towards the server consisted of a group of parties. The server then execute the protocol, and at the end, the result is returned to the client along with a verification mechanism to ensure correctness. In the fluid MPC setting, protocols are divided into three main stages: the *input stage*, the *execution stage*, and the *output stage*. During the input and output stages, communication occurs between the client and the server. The execution stage, on the other hand, involves the main computation performed by the server.

Each step of the protocol is referred to as an *epoch*. For every epoch, a group of parties is selected by the functionality $\mathcal{F}_{\text{next-com}}$, and this group is called a *committee*, denoted by $\mathcal{C}_i$ for the i -th step. The parties do not know in advance who will be part of the next committee. Only after completing their assigned step, the current committee $\mathcal{C}_i$ sends a message to the environment, and the next committee $\mathcal{C}_{i+1}$ is then determined and notified to the current participants.

Functionality $\mathcal{F}_{\text{next-com}}$

Input: (**nominate**) if a party P_i wants to join next committee

Output: $\mathcal{F}_{\text{next-com}}$ determines the next committee and informs the previous committee.

1. A party P_i inputs **nominate** and its ids to $\mathcal{Z}$ for $\mathcal{C}_{i+1}$.
2. $\mathcal{F}_{\text{next-com}}$ determines the parties for next committee and informs both the parties in $\mathcal{C}_i$ and $\mathcal{C}_{i+1}$ thorough $\mathcal{Z}$.

A fluid MPC protocol is called *maximally fluid* if the parties communicate with the next committee only once for each step. This feature makes maximally fluid MPC

an ideal version of fluid MPC, as it limits the ability of an adversary to intervene with communication. If the next committee is unknown to the current one until the final moment, it becomes much harder for an adversary to predict or disrupt the protocol. Maximally fluid MPC closely resembles the ‘You Only Speak Once’ (YOSO) framework introduced by Gentry *et al.* [19].

In the original work by Choudhuri *et al.* [15], authors design the first fluid MPC protocols for addition and multiplication under the honest majority assumption. However, later research has extended fluid MPC to support dishonest majority settings and even maximally fluid protocols, which we will discuss in the following paragraph.

As mentioned earlier, the first fluid MPC protocols were designed for the honest majority setting and did not achieve maximal fluidity. In 2022, Rachuri and Scholl introduced a protocol [20] that supports fluid MPC in the dishonest majority setting, specifically focusing on arithmetic secret sharing. Their construction rely on the SPDZ [21] and BeDoZa [22] protocols to extend the fluid MPC protocol against dishonest majority. Later, Bienstock *et al.* proposed a maximally fluid MPC protocol for the dishonest majority setting [16].

3 Technical Overview of Our Design

Before presenting $\Pi_{\text{edaBitsFluid}}$ and $\Pi_{\text{private-edaBits}}$ in the next sections, we provide here necessary functionalities and design criterias of the proposed protocols. In particular, we introduce two functionalities that are essential for supporting arithmetic operations and pairwise secret sharing, along with the general design principles underlying our constructions. The first functionality, referred to as the *Arithmetic Black Box* and denoted by $\mathcal{F}_{\text{ABB}}$, serves as a fundamental building block for our protocol.

Functionality $\mathcal{F}_{\text{ABB}}$

Parameters: All parties agree on the public parameter id_x which x is referred to data and working on $\mathbb{Z}_{2^k}$ or $\mathbb{Z}_2^k$.

Input: On input $(\text{Input}, \text{id}_x, \text{type}, x)$ from the party with the identifier id_x and input from other parties $(\text{Input}, \text{id}_y, \text{type})$, where $\text{type} \in \{\text{binary}, \text{arithmetic}\}$ and $x \in \mathbb{Z}_2^k$ or $x \in \mathbb{Z}_{2^k}$ (depending on type), store $(\text{id}_x, \text{type}, x)$ and send $(\text{Input}, \text{id}_x, \text{type})$ to $\mathcal{S}$.

ArithAdd: On input $(\text{ArithAdd}, \text{arithmetic}, \text{id}_z, \text{id}_x, \text{id}_y)$ from each party, that x, y are stored in memory, and compute $z = x + y$ where $x, y, z \in \mathbb{Z}_M$. Then stores $(\text{arithmetic}, \text{id}_z, z)$ and send $(\text{ArithAdd}, \text{arithmetic}, \text{id}_z, z)$ to $\mathcal{S}$.

BitAdd: On input $(\text{BitAdd}, \text{binary}, \text{id}_z, \text{id}_x, \text{id}_y)$ where $x = (x_0, \dots, x_{k-1}) \in \mathbb{Z}_2^k$ and $y = (y_0, \dots, y_{k-1}) \in \mathbb{Z}_2^k$ stored in memory, and compute $z = x + y$ where $x, y \in \mathbb{Z}_2^k$ and $z \in \mathbb{Z}_2^{k+\log n}$. Store $(\text{BitAdd}, \text{binary}, (z_0, \dots, z_{k+\log n-1}))$ and send $(\text{BitAdd}, \text{binary}, \text{id}_z, z)$ to $\mathcal{S}$.

Output: On input $(\text{Output}, \text{id}_z)$ from all parties (where id is present in memory), retrieve them (id_z, z) and send them to $\mathcal{S}$. Wait for an input from the $\mathcal{S}$; if this is **Deliver** then output y to all parties, otherwise output **Abort**.

Our protocol is constructed under the honest majority assumption, where each committee consists of four parties selected from a larger set of n parties for each epoch. This design choice is primarily motivated by security considerations, especially for binary shares. Unlike arithmetic shares, binary shares have not been widely explored or supported in existing fluid MPC protocols, and they need to be adapted carefully into the fluid MPC protocols to prevent the leakage against the adversary.

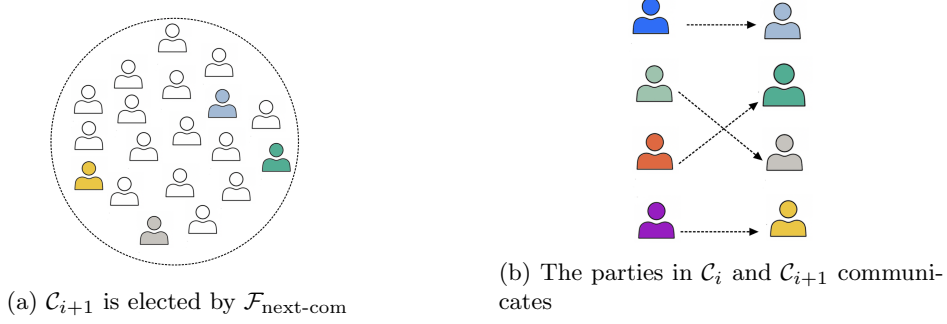


Figure 1: One epoch of a protocol

We utilize the techniques introduced in [16] and [20] extensively for sharing arithmetic secret shares with next committees. In the following paragraphs, we introduce several key functionalities, procedures, and protocols that form the foundation of our construction.

Next functionality is $\mathcal{F}_{\text{prep}}$, which plays an important role in our protocols in order to send the shares towards the next committees. This functionality provides committees with pairwise random arithmetic and binary values, which are essential for supporting subsequent procedures and protocols.

Functionality $\mathcal{F}_{\text{prep}}$

Parameters: A ring $\mathbb{Z}_{2^k}$ and parties $P_1, P_2, \dots, P_n$ and an adversary $\mathcal{A}$. We denote the set of honest parties Hon and the set of corrupted parties as Corr .

1. For each pair P_i, P_j where $i \neq j$
 - a) If both $P_i, P_j \in \text{Hon}$, $\mathcal{F}_{\text{prep}}$ samples $s_{i,j} \leftarrow \mathbb{Z}_{2^k}$.
 - b) If one the P_i or P_j is in the corrupted set, $\mathcal{F}_{\text{prep}}$ receives $s_{i,j}$ from $\mathcal{A}$.
2. For each pair P_i, P_j where $i \neq j$
 - a) If both $P_i, P_j \in \text{Hon}$, $\mathcal{F}_{\text{prep}}$ samples $t^{i,j} \leftarrow \mathbb{Z}_2$.
 - b) If one the P_i or P_j is in the corrupted set, $\mathcal{F}_{\text{prep}}$ receives $t^{i,j}$ from $\mathcal{A}$.
3. $\mathcal{F}_{\text{prep}}$ shares all computed and sampled values with $\mathcal{S}$.

4 extended daBits in Fluid MPC

The primary cryptographic primitive introduced in this work is the concept of *extended daBits* (edaBits) within the fluid MPC setting. In the previous section, we provided an overview of edaBits and their role in conventional MPC. However, there is no construction has addressed the integration of edaBits in fluid MPC. Our motivation for introducing edaBits in fluid MPC setting is to provide with flexibility for parties operating on circuits that require both arithmetic and binary computations, allowing transitions between the two domains. In the remainder of this section, we outline the key design ideas behind our construction and present the protocol $\Pi_{\text{edaBitsFluid}}$, which enables the generation and use of edaBits in the fluid MPC framework.

Functionality $\mathcal{F}_{\text{edaBitsFluid}}$

Same functionality with $\mathcal{F}_{\text{ABB}}$ with the following addition.

edaBits: On input $(\text{edaBits}, \text{id}_x)$ from each party, sample $(b_0, b_1, \dots, b_{k-1}) \in \mathbb{Z}_2^k$ uniformly and $r \in \mathbb{Z}_M$ where

$$r = \sum_{i=0}^{k-1} b_i 2^i$$

and send $(\text{edaBits}, \text{id}_x, (r, b_0, b_1, \dots, b_{k-1}))$ to $\mathcal{S}$.

The construction of edaBits in our fluid MPC protocol closely follows the edaBits protocol presented in [8], with only minor modifications. One of the differences in our design is the choice to fix the modulus as $M = 2^k$, allowing us to operate over rings. This approach builds on the ring based structures introduced and progressively refined in [?].

The reason for working over rings is as follows: when parties add their bit shares, the result can exceed k bits in length. In a field based setting, this would require an additional procedure to convert binary shares into arithmetic shares before performing further computations. In contrast, our ring based design allows a simpler solution by discarding the bits beyond the k -th position, we ensure the correctness while reducing the computational workload. As a result, working over rings enhances the efficiency of our protocol.

We now provide additional intuition regarding the concepts of *global edaBits*, *private edaBits*, and the overall *edaBits in Fluid MPC* protocol, denoted as $\Pi_{\text{edaBitsFluid}}$. As previously mentioned, private edaBits are generated through the functionality $\mathcal{F}_{\text{private-edaBits}}$, which, as the name suggests, produces secret shared values that must remain hidden from all other parties. The output of $\mathcal{F}_{\text{private-edaBits}}$ is of the form $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$. The relationship between the arithmetic and binary shares for *global edaBits* is given by $r = \sum_{i=0}^{k-1} b_i \cdot 2^i$. This equation ensures that the arithmetic value $r \in \mathbb{Z}_M$ is consistent with its binary representation.

However, when dealing with *private edaBits*, this relation may not hold directly. There is a possibility for the arithmetic and binary shares to be locally consistent, while the following may be true $[r]_M \neq \sum_{i=0}^{k-1} [b_i]_2 \cdot 2^i$. This inconsistency arises because each party receives shares of r and the b_i 's independently, and no single party can reconstruct or verify the correlation between the two components on its own. The details are given in the next section. In fact, this property is critical for maintaining the security of the protocol $\Pi_{\text{edaBitsFluid}}$. It ensures that parties cannot derive the full arithmetic or binary value only from their own private edaBits. Instead, the construction relies on the aggregation of all parties' private edaBits to form global edaBits, preserving correctness and privacy in fluid MPC setting. Reader should observe the key in understanding the protocol $\Pi_{\text{edaBitsFluid}}$ is that each party obtains private edaBits related to every other party. That is, assuming there are n parties involved, each party invokes the functionality $\mathcal{F}_{\text{private-edaBits}}$ a total of n times. As a result, each party ends up with $([r_i]_M, [b_{i,0}]_2, [b_{i,1}]_2, \dots, [b_{i,k-1}]_2)$ for $i = 1, \dots, n$.

The next step in the protocol involves aggregating these private shares. Specifically, the arithmetic components are summed modulo M , and the binary components are combined using a BitAdd operation supported by the arithmetic black box functionality $\mathcal{F}_{\text{ABB}}$.

Working over rings simplifies this process significantly. For arithmetic shares, taking the modulus $\mathbb{Z}_{2^k}$ is straightforward. Similarly, for binary shares, any overflow beyond the k -th bit can be safely discarded, as explained in the previous section. The result of this aggregation process is a set of *global edaBits*, where each party holds a share that is correctly relevant across the arithmetic and binary domains. These global edaBits can then be used as a foundational resource in other MPC applications.

Protocol $\Pi_{\text{edaBitsFluid}}$

Pre:

Parties need $\mathcal{F}_{\text{private-edaBits}}$.
Fix $M = 2^k$.

Output: Parties have $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$ such that $r = \sum_{i=0}^{k-1} b_i 2^i$.

1. Parties call $\mathcal{F}_{\text{private-edaBits}}$ functionality n times and they have $([r_i]_M, [b_{i,0}]_2, [b_{i,1}]_2, \dots, [b_{i,k-1}]_2)$ for $i = 1, \dots, n$.
2. Each party invokes the functionality $\mathcal{F}_{\text{ABB}}$ and they obtain $[r]_M = \sum_{i=1}^n [r_i]_M$.
3. Each party invokes the functionality $\mathcal{F}_{\text{ABB}}$ to BitAdd $(([b_{1,j}]_2), [b_{2,j}]_2, \dots, [b_{n,j}]_2)$, then they obtain $k + \log n$ bits for $j = 1 \dots k-1$.
4. Parties delete the bits after k -th bits then they obtain 'BitAdd'ed bits such that $[b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2$.
5. Output $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$.

The parties execute the protocol $\Pi_{\text{edaBitsFluid}}$ to obtain the global edaBits. An essential component of this protocol is the generation of private edaBits through the ideal functionality $\mathcal{F}_{\text{private-edaBits}}$, the details of which will be provided in the following

section. Before proceeding, however, we present the security proof of $\Pi_{\text{edaBitsFluid}}$ to demonstrate security guarantees under the defined adversarial model.

Theorem 1. *Let $\mathcal{A}$ be an adaptive adversary in $\Pi_{\text{edaBitsFluid}}$. Then the protocol UC-securely realizes the $\mathcal{F}_{\text{edaBitsFluid}}$ against honest majority of 4-party in the presence of $\mathcal{A}$ in the $\mathcal{F}_{\text{private-edaBits}}$ hybrid model.*

Proof We initiate by constructing the simulator $\mathcal{S}$ and an adversary $\mathcal{A}$ which is subroutine of $\mathcal{S}$. The adversary is also given access to $\mathcal{F}_{\text{private-edaBits}}$ functionality. Assume all communication is through the environment $\mathcal{Z}$. The simulator sets **status** = 1 meaning there is no abort at the beginning. If $\mathcal{A}$ cheats influencing the honest parties, the simulator updates **status** = 0. The simulation as follows.

Input: On input $\text{Input}, \text{id}_x$ simulate $\mathcal{F}_{\text{private-edaBits}}$ and obtain private edaBits. Since the private edaBits are obtained from $\mathcal{F}_{\text{private-edaBits}}$, binary and arithmetic component of the private edaBits are consistent.

ArithAdd: On input $(\text{ArithAdd}, \text{arithmetic}, \text{id}_z, \text{id}_x, \text{id}_y)$ $\mathcal{S}$ simulates the honest parties' private edaBits obtained from $\mathcal{F}_{\text{private-edaBits}}$. Even $\mathcal{A}$ cheats during this step, **status** will be 1 because there is no interaction among parties. $\mathcal{S}$ add parties' arithmetic shares locally. Furthermore, $\mathcal{A}$ cannot gain any additional information about honest parties' arithmetic shares because of the same reason, being local computation.

BitAdd: $(\text{BitAdd}, \text{binary}, \text{id}_z, \text{id}_x, \text{id}_y)$, $\mathcal{S}$ follows the same procedure as in previous step. The reason of security and leakage free is the same as well with the previous step.

Output: On input $(\text{Output}, \text{id}_z)$, $\mathcal{S}$ retrieves all parties' edaBits. Since honest parties output does not depend on the attitude of $\mathcal{A}$, $\mathcal{S}$ deliver their edaBits towards all parties.

Since all steps of the $\mathcal{F}_{\text{edaBitsFluid}}$ is local computations, $\mathcal{S}$ does not change the **status**. Therefore $\mathcal{F}_{\text{edaBitsFluid}}$ is securely realizable. $\square$

5 Private edaBits in Fluid MPC

Our main contribution is the design of the protocol for the **Private edaBits functionality**, denoted as $\mathcal{F}_{\text{priv-edaBits}}$, adapted for the fluid MPC setting. We will formally define this functionality and present its security model in the subsequent sections.

5.1 Security Model

We design a new functionality, denoted as $\mathcal{F}_{\text{priv-edaBits}}$, to model private edaBits generation within the fluid MPC setting. Our protocol achieves *security with abort*, meaning that if honest parties detect any misbehavior or inconsistent information, the protocol is interrupted to prevent the transmission of incorrect data to the clients.

The functionality is defined over the ring $\mathbb{Z}_{2^k}$, supporting both binary and arithmetic resharing between committees.

Functionality $\mathcal{F}_{\text{private-edaBits}}$

Parameters: All parties and the clients agree on the public parameter id_x which x is referred to data and working on the ring $\mathbb{Z}_{2^k}$.

Init: On input $(\text{Init}, \mathcal{C})$ from each party $P_j \in \mathcal{C}_{clnt}$, where each party sending the same set of parties. Then they initialize the the first committee $\mathcal{C}_1$ and send $(\text{Init}, \mathcal{C}_1)$ to $\mathcal{S}$.

Input: On input $(\text{Input}, \text{id}_x, x)$ from the parties, $P_i \in \mathcal{C}_i$ and input $(\text{Input}, \text{id}_x)$ from other parties. It stores (id_x, x) and sends $(\text{Input}, \text{id}_x)$ to $\mathcal{S}$.

Binary Bucket Sum: On input $(\text{BBS}, \text{id}_x, \text{id}_y, \text{id}_z)$ from each party $P_i \in \mathcal{C}_i$, compute $z = x + y \in \mathbb{Z}_2^k$. It stores (id_z, z) and sends $(\text{BBS}, \text{id}_z)$ to $\mathcal{S}$.

Arithmetic Bucket Sum: On input $(\text{ABS}, \text{id}_x, \text{id}_y, \text{id}_z)$ from each party $P_i \in \mathcal{C}_i$, compute $z = x + y \in \mathbb{Z}_{2^k}$. It stores (id_z, z) and sends $(\text{ABS}, \text{id}_z)$ to $\mathcal{S}$.

Check: On input $(\text{check}, \text{id}_z)$ from the $\mathcal{C}_{clnt}$ for a value z with its id_z has been stored in previous steps, it sends $(\text{check}, \text{id}_z, z)$ to $\mathcal{S}$. If $\mathcal{S}$ does not abort it sends the the output to $\mathcal{C}_{clnt}$.

Escudero *et al.* construct the private edaBits protocol under the assumption of static parties in a non-fluid setting in [8]. In their approach, each party sends a number of binary shares along with their corresponding arithmetic encodings to other parties. The protocol relies on a cut-and-choose technique to verify the consistency between the binary and arithmetic components of the edaBits. As part of this technique, the *bucket method* is employed. The bucket method, in short, involves categorizing the shared values into groups known as *buckets*. Specifically, each party sends a set of binary and arithmetic shares to the other parties. The recipients then place the received edaBits into buckets. Within each bucket, the edaBits are added sequentially, and the resulting aggregated values are subsequently opened for verification. If the binary and arithmetic parts match as expected, the edaBits are accepted as consistent and kept for further use. We refer the reader to [24] and [25] for standard cut-and-choose techniques, and to [8] for a variant adapted for edaBits.

However, in the fluid MPC setting, such a direct approach is not valid. Since different parties may be included in each epoch, directly transmitting both the binary and arithmetic components of the edaBits would risk leaking information. Despite this limitation, we adapt the bucket based verification in a modified form suitable for the fluid MPC model.

The work [16] utilizes the **next-committee functionality** from [15]. One of the key distinctions in our construction lies in the design and use of the next-committee functionality, denoted as $\mathcal{F}_{\text{next-com-priv}}$, which builds upon the ideas introduced in [15, 16]. The motivation for this modification is about the challenge of securely transferring binary secret shares to the next committee in a fluid MPC setting, particularly because our protocol requires the secure resharing of both binary and arithmetic shares between committees.

To mention this, our first design decision is to reduce the number of parties in each committee. Specifically, $\mathcal{F}_{\text{next-com-priv}}$ chooses exactly four parties for each committee. Secondly, the functionality is designated for an honest-majority setting. That is, $\mathcal{F}_{\text{next-com-priv}}$ ensures that each committee includes at least three honest parties in every epoch. This structure forms the basis for our protocol's next-committee functionality, which is designed to fulfill the needs of the private edaBits protocol in a fluid MPC environment.

Functionality $\mathcal{F}_{\text{next-com-priv}}$

Input: (Next-Committee) from all $P_i \in \mathcal{C}_i$

Output: $(P'_x, P'_y, P'_z, P'_t \in \mathcal{C}_{i+1}, \text{ and } \text{id}_{P'_x}, \text{id}_{P'_y}, \text{id}_{P'_z}, \text{id}_{P'_t})$

1. Parties P'_x, P'_y, P'_z, P'_t input **nominate** and their $\text{id}_{P'_x}, \text{id}_{P'_y}, \text{id}_{P'_z}, \text{id}_{P'_t}$ for $\mathcal{C}_{i+1}$.
2. $\mathcal{F}_{\text{next-com-priv}}$ determines and gives the id of $P'_x, P'_y, P'_z, P'_t \in \mathcal{C}_{i+1}$ towards parties $P_x, P_y, P_z, P_t \in \mathcal{C}_i$ through $\mathcal{Z}$.

Upon receiving these shares, each party constructs a bucket, placing one edaBit at the top, designated as the candidate private edaBit. The remaining edaBits in the bucket are used for consistency checks. Specifically, each party adds the binary part of the top edaBit to each of the remaining binary parts, and performs the same addition on the arithmetic parts. Then, these added shares are exchanged among the parties and opened. The arithmetic and binary results are compared for consistency. If all comparisons pass, the top edaBit is accepted as a valid private edaBit and it is candidate to be used in the protocol Π_{edaBits} .

Our second major distinction arises in the design of the bucket method. Unlike the approach in [8], where the consistency of each edaBits' binary and arithmetic components is verified individually, we adopt a more aggregated strategy. Specifically, we construct two separate buckets: one for binary shares and another for arithmetic shares. Instead of performing pairwise checks, we sum all elements within each bucket once for the binary shares and once for the arithmetic shares.

At the end of the protocol, each party obtains the aggregate sum of both buckets and checks whether the two totals are consistent according to the edaBits relation. This collective summation significantly reduces the communication and computation cost. Although the bucket size is configurable, we fix it to 8 in the protocol $\Pi_{\text{private-edaBits}}$ presented below.

In the following two sections we first introduce how to send and bucket sum for binary and arithmetic parts of the edaBits respectively.

5.2 Binary Part of $\Pi_{\text{private-edaBits}}$

In this section, we give the details of the procedure for transferring binary shares to the next committees and explain how the aggregated bucket sum of these shares is computed.

MPC in the fluid setting for arithmetic shares has been extensively studied in [15], [20], and [16]. However, binary shares have received less attention in the context of fluid MPC. In particular, edaBits require the simultaneous proceeding of both binary and arithmetic shares. This requires a secure mechanism for transferring binary shares across committees, which motivates the need for a new construction that enables such secure resharing of binary data in the fluid setting.

For the binary part of the protocol $\Pi_{\text{private-edaBits}}$, the parties in the committee $\mathcal{C}_i$ each hold a single edaBits excluding the arithmetic share, consisting of binary shares $[b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2$. For simplicity, we denote such a binary-only edaBits as r^i . To clarify for the reader, the client committee $\mathcal{C}_{\text{clnt}}$ uniformly samples all bits of r^i for

$i = 1, 2, 3, 4$, corresponding to each party in the committee. The global binary part of the edaBits is then computed as the sum $r = r^1 + r^2 + r^3 + r^4$.

We first introduce a procedure, denoted as $\pi_{\text{binary-reshare}}$, which is a modified version of the $\pi_{\text{arithmetic-reshare}}$ protocol presented in [8]. In $\pi_{\text{binary-reshare}}$, parties transfer binary secret shares to the parties in the next committee for each bit, using pairwise random bits provided by the functionality $\mathcal{F}_{\text{prep}}$ and applying bit addition. Through this procedure, binary secrets can be securely reshared and reconstructed via bit addition. The detailed steps of the procedure are given below.

Procedure $\pi_{\text{binary-reshare}}$

Usage: Parties in $\mathcal{C}_i$ reshare binary shares $\{[b_0], [b_1], \dots, [b_{k-1}]\}$ towards the parties in $\mathcal{C}_{i+1}$

We present below the step-by-step procedure for how the parties reshare their values corresponding to the first bit.

1. Parties in $\mathcal{C}_i$ invokes $\mathcal{F}_{\text{prep}}$ and obtains the fresh pairwise random values $\{t_{j,l}\}_{l \in \mathcal{C}_{i+1}}$ for the first bit $[b_0]$.
2. Each party $P_j \in \mathcal{C}_i$ has the first bit of binary shares $[b_0]$ as $b^j \in \{0, 1\}$. They computes

$$t^{j,1} = b^j - \sum_{l=2}^n t_{j,l}$$

by using BitAdd.

3. Each P_j sends their $t^{j,1}$ to $P_1 \in \mathcal{C}_{i+1}$.
4. Parties P_l in $\mathcal{C}_{i+1}$ computes

$$b^l = \sum_{j \in \mathcal{C}_i} t_{j,l},$$

where if $l \neq 1, t^{j,l} = t_{j,l}$ obtained from $\mathcal{F}_{\text{prep}}$.

Then parties follow the same procedure for each bit such that $\{[b_0], [b_1], \dots, [b_{k-1}]\}$.

Lemma 1. *The transcript of $\pi_{\text{binary-reshare}}$ is simulatable by random values against dishonest majority.*

Proof Without loss of generality, assume the worst case scenario, there is only one party $P_j \in \mathcal{C}_i$ is the honest party in the committee $\mathcal{C}_i$. P_j has to communicate only $P_1 \in \mathcal{C}_{i+1}$, other pairwise binary secret shares $t_{j,l}$ is distributed by $\mathcal{F}_{\text{prep}}$. If P_1 is not corrupted in the committee $\mathcal{C}_{i+1}$, then the procedure is simulatable because binary secret share is hid inside $t^{j,1}$ and P_1 just follow the protocol. Now, assume $P_1 \in \mathcal{C}_{i+1}$ is corrupted and without loss of generality $P_l \in \mathcal{C}_{i+1}$ is honest. Since P_1 cannot know the pairwise binary secret share $t_{j,l}$ distributed by $\mathcal{F}_{\text{prep}}$ to P_j and P_l , where $j, l \neq 1$, and $t_{j,l}$ is the additive component of $t^{j,1}$,

$t_{j,l}$ seems uniformly random towards the adversary, $\mathcal{A}$ and $\mathcal{A}$ has to gain of knowledge about $t_{j,l}$ to obtain $[b_j]_2$. $\square$

We now introduce the second procedure, called $\pi_{\text{binary-double-encryption}}$, which performs an additional encryption layer over the binary secret shares. Since the binary part of the edaBits appears indistinguishable from random to the client $\mathcal{C}_{\text{clnt}}$, this step is necessary to enable $\mathcal{C}_{\text{clnt}}$ to verify not only the consistency but also the correctness of the binary secret shares at the end of the protocol. To enable this verification, $\mathcal{C}_{\text{clnt}}$ must possess an encrypted version of each bit. Therefore, the binary secret shares are double-encrypted using this procedure.

In $\pi_{\text{binary-double-encryption}}$, each party arranges their binary part of the edaBits by uniformly sampling bits. Given that r^1 represents the edaBit placed at the top of the bucket, the party samples k bits independently, four times, in such a way that the bit addition of these sampled shares equals r^1 . The detailed procedure is described below.

Procedure $\pi_{\text{binary-double-encryption}}$

Input: $\mathcal{C}_{\text{clnt}}$ has the binary component of private edaBits r^1 .

Usage: In order to verify the integrity of r^1 .

1. $\mathcal{C}_{\text{clnt}}$ arrange the binary shares as following

$$r = r^1 + r^2 + r^3 + r^4.$$

where $r^1, r^2, r^3, r^4 \in \mathbb{Z}_2$ and $+$ is denoted for BitAdd inside $\mathcal{F}_{\text{ABB}}$.

2. $\mathcal{C}_{\text{clnt}}$ arrange all the arithmetic shares for double encryption as following

$$r^1 = \alpha_{r^1} + \beta_{r^1} + \gamma_{r^1} + \theta_{r^1}.$$

,where $\alpha_{r^1}, \beta_{r^1}, \gamma_{r^1}, \theta_{r^1} \in \{0, 1\}^k$.

3. $\mathcal{C}_{\text{clnt}}$ invokes $\pi_{\text{binary-reshare}}$ for $\alpha_{r^1}, \beta_{r^1}, \gamma_{r^1}, \theta_{r^1}$.
4. Parties in the epoch until the $\mathcal{C}_{\text{clnt}}$ invokes $\pi_{\text{binary-reshare}}$ and in the last epoch each share is opened to $\mathcal{C}_{\text{clnt}}$.

Lemma 2. *Procedure $\pi_{\text{binary-double encryption}}$'s transcript is simulatable against honest majority of 4-party.*

Proof Without loss of generality, assume the worst case scenario which is in both committees there exist a corrupted party such that $P_j \in \mathcal{C}_i$ and $P_1 \in \mathcal{C}_{i+1}$. Since the double encrypted binary shares are shared among all parties by $\pi_{\text{binary-reshare}}$ which is simulatable as showed above, corrupted parties can just see one doubly encrypted binary share, wlog let's say α_{r^1} 's binary share from binary reshare, so, corrupted parties need all binary shares of α_{r^1} and it is one component of r^1 . Furthermore, even corrupted parties being in consecutive committees share their secret binary shares they cannot obtain α_{r^1} because they need two more binary secrets. Therefore, all binary shares obtained by invoking $\pi_{\text{binary-reshare}}$ seems uniformly random towards $\mathcal{A}$ and $\mathcal{A}$ has to gain of knowledge about two more binary share to obtain α_{r^1} which is impossible for 4-party honest majority system. $\square$

The following procedure is denoted as $\pi_{\text{binary-bucket-sum}}$, in which the parties compute the sum of each binary secret element within the bucket. At the beginning of the protocol, $\mathcal{C}_{clnt}$ uniformly samples r^1, r^2, r^3, r^4 , which are binary private edaBits assigned to each party, and also samples camouflage bits c^1, c^2, c^3, c^4 uniformly such that $c^1 + c^2 + c^3 + c^4 \in \mathbb{Z}_2^k$, which are elements of the bucket.

Subsequently, $\mathcal{C}_{clnt}$ arranges the binary secret shares as $(2r^1 - r^2 + c^1)$, $(2r^2 - r^3 + c^2)$, $(2r^3 - r^4 + c^3)$, and $(2r^4 - r^1 + c^4)$, in order to hide and share the secret binary values r^1, r^2, r^3 , and r^4 . Each expression of the form $2r^i - r^j + c^i$ is referred to as a **bit triple** for simplicity. The binary components of the edaBits can be distributed among the parties in $\mathcal{C}_i$ by $\mathcal{C}_{clnt}$. For the remainder of this section, we refer to r^1 as if it represents the binary share of the top edaBit. However, this does not imply that it is definitively fixed or transparently revealed to the parties. It is simply one of the edaBits determined by $\mathcal{C}_{clnt}$ and may occupy any position within the bucket.

You can see the details of the procedure is below, but we want to explain how $\pi_{\text{binary-bucket-sum}}$ works. The core idea behind the operation of $\pi_{\text{binary-bucket-sum}}$ explains the choice of having four parties and requiring an honest majority. Specifically, it is assumed that in each epoch, at most one party may be corrupted. If bit triples are verified by two different parties, then since at least one of them is honest, so any incorrect bit triple can be detected. This principle underlies the protocol structure: in the first epoch, each party generates two bit triples and sends the corresponding bit triples to the appropriate parties in the second epoch. As a result, in the second epoch, each party receives identical bit triples from two different parties who participated in the first epoch.

Parties then verify the consistency of the received bit triples. If any inconsistency is detected, the protocol is aborted. Otherwise, if all bit triples are consistent, the parties proceed to invoke $\mathcal{F}_{ABB}$ for bit addition, denoted as BitAdd. The communication between the second and third epochs follows a similar pattern. However, in this step, the parties send the added bit triples, along with the corresponding original bit triples, to the parties in the third epoch. When the parties invoke $\mathcal{F}_{ABB}$ for bit addition, BitAdd, they obtain the sum

$$r^1 + r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4 = r + c \in \mathbb{Z}_2^k,$$

expressed in binary form.

In addition to computing $r + c \in \mathbb{Z}_2^k$, the procedure $\pi_{\text{binary-bucket-sum}}$ also involves that $\mathcal{C}_{clnt}$ performs an additional bit addition, at the beginning, to obtain the sum

$$r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4.$$

This bucket sum, which excludes r^1 , is utilized in subsequent procedures to verify the correctness of the binary component of the edaBits, r^1 . We give more details in the security analysis of $\pi_{\text{binary-bucket-sum}}$.

Procedure $\pi_{\text{binary-bucket-sum}}$

Input: $\mathcal{F}_{\text{ABB}}$ with modulus $M = 2^k$ and length k where $k < \log_2^M$

Output: A party has shared private edaBits $([r_j]_M, [b_{j,0}]_2, [b_{j,1}]_2, \dots, [b_{j,k-1}]_2)$

1. $\mathcal{C}_{\text{clnt}}$ samples $\{(b_{i,0}, b_{i,1}, \dots, b_{i,k-1})\}_{i=1}^8 \in \mathbb{Z}_2^k$ where the first forth k -th bits $\{(b_{i,0}, b_{i,1}, \dots, b_{i,k-1})\}_{i=1}^4$ is r^1, r^2, r^3 and r^4 respectively and the second fourth k -th bits are $\{(b_{i,0}, b_{i,1}, \dots, b_{i,k-1})\}_{i=5}^8$ is c^1, c^2, c^3, c^4 for camouflage bit part of private edaBits.
2. $\mathcal{C}_{\text{clnt}}$ invokes $\mathcal{F}_{\text{prep}}$ for bit addition, BitAdd, to obtain $(2r^1 - r^2 + c^1), (2r^2 - r^3 + c^2), (2r^3 - r^4 + c^3), (2r^4 - r^1 + c^4)$ and $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ and invokes $\pi_{\text{binary-reshare}}$ on it (and $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ is invoked $\pi_{\text{binary-reshare}}$ until last epoch).
3. $\mathcal{C}_{\text{clnt}}$ shares the bit and arithmetic shares

$$\begin{aligned} &(2r^1 - r^2 + c^1, 2r^2 - r^3 + c^2) \text{ with } P_1, \\ &(2r^2 - r^3 + c^2, 2r^3 - r^4 + c^3) \text{ with } P_2, \\ &(2r^3 - r^4 + c^3, 2r^4 - r^1 + c^4) \text{ with } P_3, \\ &(2r^4 - r^1 + c^4, 2r^1 - r^2 + c^1) \text{ with } P_4. \end{aligned}$$
4. Parties P_1, P_2, P_3, P_4 in $\mathcal{C}_i$ shares their combined binary shares towards the parties P'_1, P'_2, P'_3, P'_4 in $\mathcal{C}_{i+1}$ as following;

$$\begin{aligned} P_1 &\text{ sends } 2r^1 - r^2 + c^1 \text{ and } 2r^2 - r^3 + c^2 \text{ to } P'_1 \\ P_2 &\text{ sends } 2r^2 - r^3 + c^2 \text{ and } 2r^3 - r^4 + c^3 \text{ to } P'_2 \\ P_3 &\text{ sends } 2r^3 - r^4 + c^3 \text{ and } 2r^4 - r^1 + c^4 \text{ to } P'_3 \\ P_4 &\text{ sends } 2r^4 - r^1 + c^4 \text{ and } 2r^1 - r^2 + c^1 \text{ to } P'_4 \end{aligned}$$
5. Parties, also, send control bit shares as following;

$$\begin{aligned} P_1 &\text{ sends } (2r^2 - r^3 + c^2) \text{ to } P'_2, (2r^1 - r^2 + c^1) \text{ to } P'_3 \text{ and } (2r^1 - r^2 + c^1), \\ &\quad (2r^2 - r^3 + c^2) \text{ to } P'_4. \\ P_2 &\text{ sends } (2r^2 - r^3 + c^2), (2r^3 - r^4 + c^3) \text{ to } P'_1, (2r^3 - r^4 + c^3) \text{ to } P'_3 \text{ and} \\ &\quad (2r^2 - r^3 + c^2) \text{ to } P'_4. \\ P_3 &\text{ sends } (2r^3 - r^4 + c^3) \text{ to } P'_1, (2r^3 - r^4 + c^3), (2r^4 - r^1 + c^4) \text{ to } P'_2 \text{ and} \\ &\quad (2r^4 - r^1 + c^4) \text{ to } P'_4. \\ P_4 &\text{ sends } (2r^1 - r^2 + c^1) \text{ to } P'_1, (2r^4 - r^1 + c^4) \text{ to } P'_2 \text{ and } (2r^1 - r^2 + c^1), \\ &\quad (2r^4 - r^1 + c^4) \text{ to } P'_3. \end{aligned}$$
6. Each party checks their control bits sent by two different parties, if all control bits are consistent, they continue.
7. Parties $P'_1, P'_2, P'_3, P'_4 \in \mathcal{C}_{i+1}$ locally BitAdd by using $\mathcal{F}_{\text{ABB}}$, then P'_1 obtains $2r^1 + r^2 + r^3 - r^4 + c^1 + c^2 + c^3$, P'_2 obtains $-r^1 + 2r^2 + r^3 + r^4 + c^2 + c^3 + c^4$, P'_3 obtains $r^1 - r^2 + 2r^3 + r^4 + c^1 + c^3 + c^4$ and P'_4 obtains $r^1 + r^2 - r^3 + 2r^4 + c^1 + c^2 + c^4$.
8. Parties $P'_1, P'_2, P'_3, P'_4 \in \mathcal{C}_{i+1}$, sends bit shares obtained from previous step to $P''_1, P''_2, P''_3, P''_4$ where $P''_1, P''_2, P''_3, P''_4 \in \mathcal{C}_{i+2}$. At this stage the parties $\in \mathcal{C}_{i+1}$ send control bit shares as following;

P'_1 sends $2r^1 - r^2 + c^1$ to P''_2 , $2r^2 - r^3 + c^2$ to P''_3 and $2r^3 - r^4 + c^3$ to P''_4
 P'_2 sends $2r^4 - r^1 + c^4$ to P''_1 , $2r^2 - r^3 + c^2$ to P''_3 and $2r^3 - r^4 + c^3$ to P''_4
 P'_3 sends $2r^4 - r^1 + c^4$ to P''_1 , $2r^1 - r^2 + c^1$ to P''_2 and $2r^3 - r^4 + c^3$ to P''_4
 P'_4 sends $2r^4 - r^1 + c^4$ to P''_1 , $2r^1 - r^2 + c^1$ to P''_2 and $2r^2 - r^3 + c^2$ to P''_3

9. Each party checks their control bits sent by two different parties, if all control bits are consistent, they continue.
10. Parties $\in \mathcal{C}_{i+2}$ locally BitAdd by using $\mathcal{F}_{\text{ABB}}$, then all parties obtains $r^1 + r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$.
11. Parties $\in \mathcal{C}_{i+2}$ have $r^1 + r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4 = r + c \in \mathbb{Z}_2^k$ and they sent to this binary sum to the relevant party $\in \mathcal{C}_{\text{clnt}}$. Additionally parties in $\mathcal{C}_{i+2}$ open the binary secret shares of $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ obtained from $\pi_{\text{binary-reshare}}$ towards $\mathcal{C}_{\text{clnt}}$.

Security of $\pi_{\text{binary-bucket-sum}}$

In this part we shall mention about security of $\pi_{\text{binary-bucket-sum}}$ and we explain why we choose four party honest majority setting for private edaBits. We consider the security case by case.

As discussed above, the underlying principle of $\pi_{\text{binary-bucket-sum}}$ is that honest parties can verify the consistency of the binary shares. Even if one party is corrupted, any manipulated share by the corrupted party is also processed by another honest party. Given that the protocol assumes at most one corrupted party, at least one copy of each binary share must be correct. If a party receives two different versions of a binary share, it can deduce that one of the parties in the previous committee is corrupted and will subsequently abort the protocol. If all binary shares are consistent, the protocol concludes successfully, and the corresponding client obtains their private edaBits.

We have four epochs for $\pi_{\text{binary-bucket-sum}}$. In the first epoch client shares the arithmetic and binary shares to parties in the first committee $\mathcal{C}_1$ by invoking the procedures $\pi_{\text{binary-reshare}}$ and $\pi_{\text{binary-double-encryption}}$. Furthermore, $\mathcal{C}_{\text{clnt}}$ arranges the bit triples as $2r^i - r^j + c^i$ by combining the binary of the edaBits and camouflage bits for each party in $\mathcal{C}_1$.

In the first case, without loss of generality, consider the worst case scenario where exactly one party is corrupted in each committee. Due to the design of our protocol, he is not possible for two parties within the same committee to be corrupted at the same time. For simplicity, we denote the bit triples by the following notation: $\mathbf{a} := 2r^1 - r^2 + c^1$, $\mathbf{b} := 2r^2 - r^3 + c^2$, $\mathbf{c} := 2r^3 - r^4 + c^3$, and $\mathbf{d} := 2r^4 - r^1 + c^4$. Assume that the fourth party in each committee is corrupted.

In $\mathcal{C}_1$, suppose P_4 manipulates with its binary shares $\mathbf{a}$ and $\mathbf{d}$. Through the mechanism $\pi_{\text{binary-bucket-sum}}$, P_4 transmits manipulated bit triples to each party in $\mathcal{C}_2$. Since $P'_4 \in \mathcal{C}_2$ is also corrupted, we can temporarily ignore his behavior. Now consider $P'_2 \in \mathcal{C}_2$, who possesses three correct binary shares $\mathbf{b}$, $\mathbf{c}$, and one potentially manipulated binary share $\mathbf{d}^*$ received from P_4 . However, P'_2 also receives another instance of $\mathbf{d}$ from P_3 . If $\mathbf{d}^* \neq \mathbf{d}$, then P'_2 aborts the protocol.

A similar argument applies for all cases occurring in the second epoch. The use of control bit triples allows us to securely transfer towards the next epoch. In the third epoch, it is guaranteed that all honest parties hold valid binary shares, under the assumption that P'_4 is corrupted. During this epoch, the parties perform bit addition by invoking $\mathcal{F}_{\text{ABB}}$, resulting in the following values: P'_1 computes $\mathbf{a} + \mathbf{b} + \mathbf{c}$, P'_2 computes $\mathbf{b} + \mathbf{c} + \mathbf{d}$, P'_3 computes $\mathbf{a} + \mathbf{c} + \mathbf{d}$, and P'_4 computes $\mathbf{a} + \mathbf{b} + \mathbf{d}$.

Each party in $\mathcal{C}_2$ transmits the sum of its binary shares to the party with the corresponding index in $\mathcal{C}_3$. It is important to observe that each party needs exactly one binary share, which is subsequently provided by all other parties with differing indices in $\mathcal{C}_i$. Thus, even if P'_4 transmits a manipulated share to P''_1 , he can detect the inconsistency, as he also receives two additional bit triples from honest parties.

Next, we consider an alternative scenario involving the potential leakage of the binary component of r . Suppose that the adversary $\mathcal{A}$ obtains all the bit triples, namely $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$, and $\mathbf{d}$. This situation may arise, for instance, if $P_1 \in \mathcal{C}_1$ and $P_3 \in \mathcal{C}_2$ are corrupted.

Nevertheless, due to the camouflage bits, $\mathcal{A}$ is unable to recover the binary secret. Although $\mathcal{A}$ possesses four bit triples, there are eight unknowns in total. Consequently, the adversary only has access to four equations in eight variables, which is insufficient to solve for the binary representation of r . Therefore, even in the event that $\mathcal{A}$ obtains all bit triples, no meaningful information regarding the binary form of r can be extracted.

Therefore, we can see that the procedure $\pi_{\text{binary-bucket-sum}}$ has no leakage towards any corrupted parties and the adversary $\mathcal{A}$. Additionally, if any corrupted party manipulates the bit triples, honest parties abort the protocol and can detect the corrupted party. Finally, we provide a justification for our chosen construction. While it is indeed possible to aggregate the binary components of the private edaBits using the procedures $\pi_{\text{binary-double-encryption}}$ and $\pi_{\text{binary-reshare}}$, doing so would require additional cryptographic structures such as SPDZ [21], BeDoZa [22], or the accumulator mechanism introduced in [16]. In contrast, our proposed procedure, $\pi_{\text{binary-bucket-sum}}$ eliminates the dependency on such protocols and cryptographic primitives. The use of $\pi_{\text{binary-bucket-sum}}$ alone suffices to ensure the correctness of the computed bucket sum.

Lemma 3. *Procedure $\pi_{\text{binary-bucket-sum}}$'s transcript is simulatable by random values against honest majority of 4-party.*

Proof Without loss of generality, consider the worst-case scenario in which a corrupted party appears in each consecutive epoch and is positioned such that the adversary $\mathcal{A}$ can access all bit triples. Since each binary component of the private edaBits is hidden using independently sampled camouflage bits, the bit triples appear uniformly random to $\mathcal{A}$. Even if $\mathcal{A}$ obtains all of the bit triples, it cannot learn the value of r^1 , as the number of unknowns exceeds the number of equations represented by the bit triples. Consequently, the bit triples, used to mask the binary components, r^1 , of private edaBits remain indistinguishable from random values towards the adversary $\mathcal{A}$ and can be simulated accordingly. $\square$

5.3 Arithmetic Part of $\Pi_{\text{private-edaBits}}$

In this section we give the details of how the arithmetic shares are transferred to the next committees and how their bucket sum is obtained.

In $\mathcal{C}_i$ parties have one edaBits excluded binary shares such that $[r]_M$. For simplicity, we call r_i for one edaBits without its binary shares.

We begin by introducing the procedure $\pi_{\text{arithmetic-reshare}}$, which is used for the arithmetic component of $\Pi_{\text{private-edaBits}}$. This procedure employs the technique proposed in [20]. Specifically, the parties in $\mathcal{C}_i$ transfer their additive arithmetic shares $[r]_M$ to the parties in $\mathcal{C}_{i+1}$ using pairwise random values generated via the functionality $\mathcal{F}_{\text{prep}}$. Once all arithmetic secret shares have been revealed, the client $\mathcal{C}_{\text{clnt}}$ can reconstruct the value r . The detailed description of the procedure is presented below.

Procedure $\pi_{\text{arithmetic-reshare}}$

Usage: $\mathcal{C}_i$ reshapes their arithmetic shares $[r]_M$ to $\mathcal{C}_{i+1}$

1. Parties in $\mathcal{C}_i$ invokes $\mathcal{F}_{\text{prep}}$ and obtains the fresh pairwise random values $\{s^{j,l}\}_{l \in \mathcal{C}_{i+1}}$.
2. Each party $P_j \in \mathcal{C}_i$ has an arithmetic share r_j . They computes

$$s_{j,1} = r_j - \sum_{l=2}^n s^{j,l}$$

3. Each P_j sends their $s_{j,1}$ to $P_1 \in \mathcal{C}_{i+1}$.
4. Parties P_j in $\mathcal{C}_{i+1}$ computes

$$s_j = \sum_{l \in \mathcal{C}_i} s^{j,l}$$

Lemma 4. *The transcript of $\pi_{\text{arithmetic-reshare}}$ is simulatable by random values against dishonest majority.*

Proof Without loss of generality, assume the worst case scenario, there is only one party $P_j \in \mathcal{C}_i$ is the honest party in the committee $\mathcal{C}_i$. P_j has to communicate only $P_1 \in \mathcal{C}_{i+1}$, other pairwise secret shares $s^{j,l}$ is distributed by $\mathcal{F}_{\text{prep}}$. If P_1 is not corrupted in the committee $\mathcal{C}_{i+1}$, then the procedure is simulatable because arithmetic secret share is hidden inside $s_{j,1}$ and P_1 just follow the protocol. Now, assume $P_1 \in \mathcal{C}_{i+1}$ is corrupted and without loss of generality $P_l \in \mathcal{C}_{i+1}$ is honest. Since P_1 cannot know the pairwise secret share $s^{j,l}$ distributed by $\mathcal{F}_{\text{prep}}$ to P_j and P_l and $s^{j,l}$ is the additive component of $s_{j,1}$, $s^{j,l}$ seems uniformly random towards the adversary, $\mathcal{A}$ and $\mathcal{A}$ has to gain of knowledge about $s^{j,l}$ to obtain r_j . $\square$

Next, we introduce the second procedure for the arithmetic component, denoted as $\pi_{\text{arithmetic-double-encryption}}$, which follows the same principle as $\pi_{\text{binary-double-encryption}}$.

The purpose of $\pi_{\text{arithmetic-double-encryption}}$ is to apply a second layer of additive encryption to the arithmetic secret shares. This additional encryption is necessary because, at the conclusion of the protocol, the client $\mathcal{C}_{\text{clnt}}$ must verify not only the consistency of the arithmetic shares with the binary component but also their correctness. To handle this verification, $\mathcal{C}_{\text{clnt}}$ must possess the arithmetic component of the edaBits and this component should not be obtained by other parties. Therefore, we apply a double encryption to the arithmetic secret shares to ensure their integrity and verifiability.

In the procedure $\pi_{\text{binary-double-encryption}}$, each party prepares the arithmetic component of their edaBits. Although r_1 is identified as the edaBit located at the top of the bucket, the party must apply the double encryption procedure to all values r_1, r_2, r_3 , and r_4 . To accomplish this, the party samples a total of 16 arithmetic values such that their sums correspond to the values r_1, r_2, r_3 , and r_4 , respectively. This ensures that each r_i is represented as the sum of four random additive shares, thereby enabling secure double encryption of the arithmetic components. The detailed steps of the procedure are provided below.

Procedure $\pi_{\text{arithmetic-double-encryption}}$

Input: $\mathcal{C}_{\text{clnt}}$ has the arithmetic share r .

Usage: In order to obtain bucket sum $\mathcal{C}_{\text{clnt}}$ double encrypts the arithmetic share.

1. $\mathcal{C}_{\text{clnt}}$ arrange the arithmetic shares as following

$$r = r_1 + r_2 + r_3 + r_4.$$

where $r_1, r_2, r_3, r_4 \in \mathbb{Z}_{2^k}$

2. $\mathcal{C}_{\text{clnt}}$ arrange all the arithmetic shares for double encryption as following

$$r_1 = \alpha_{r_1} + \beta_{r_1} + \gamma_{r_1} + \theta_{r_1}$$

$$r_2 = \alpha_{r_2} + \beta_{r_2} + \gamma_{r_2} + \theta_{r_2}$$

$$r_3 = \alpha_{r_3} + \beta_{r_3} + \gamma_{r_3} + \theta_{r_3}$$

$$r_4 = \alpha_{r_4} + \beta_{r_4} + \gamma_{r_4} + \theta_{r_4}$$

where $\{\alpha_{r_i}, \beta_{r_i}, \gamma_{r_i}, \theta_{r_i}\}_{i=1}^4 \in \mathbb{Z}_{2^k}$.

Lemma 5. *Procedure $\pi_{\text{arithmetic-double encryption}}$'s transcript is simulatable against honest majority of 4-party.*

Proof The adversary $\mathcal{A}$ communicates with $\pi_{\text{arithmetic-reshare}}$'s values and the values are simulatable by random values from Lemma 3. Furthermore, there is no interaction among parties when $\mathcal{C}_{\text{clnt}}$ arranges the doubly encrypted shares, they are arranged locally. Thus $\pi_{\text{arithmetic-double-encryption}}$ is also simulatable for 4-party honest majority setup. $\square$

The next procedure for the arithmetic component is denoted as $\pi_{\text{arithmetic-bucket-sum}}$, which computes the bucket sum of the arithmetic parts of all edaBits. At the conclusion of this procedure, the client $\mathcal{C}_{\text{clnt}}$ receives two outputs:

the full bucket sum of the arithmetic components and the **diminished arithmetic bucket sum**, which refers to the bucket sum with the value corresponding to r_1 subtracted. The latter is used specifically to verify the correctness of r_1 .

In the procedure $\pi_{\text{arithmetic-bucket-sum}}$, two different arithmetic shares are selected, and the protocol $\pi_{\text{arithmetic-double-encryption}}$ is invoked on each of them. Subsequently, the corresponding doubly encrypted shares are added together. From this sum, a diminished version, excluding the contribution from r_1 , is also computed. The detailed steps of the procedure are provided below.

Procedure $\pi_{\text{arithmetic-bucket-sum}}$

Input: Arithmetic shares r and c .

Usage: Deliver the arithmetic parts of the edaBits with arithmetic part of bucket sum.

1. $\mathcal{C}_{clnt}$ invokes $\pi_{\text{arithmetic-double-encryption}}$ for r and c which are arithmetic secret shares.
2. Then $\mathcal{C}_{clnt}$ arrange new summation for arithmetic bucket some as following;

$$\mathcal{X} = \alpha_{r_1} + \alpha_{r_2} + \alpha_{r_3} + \alpha_{r_4} + \alpha_{c_1} + \alpha_{c_2} + \alpha_{c_3} + \alpha_{c_4}$$

$$\mathcal{Y} = \beta_{r_1} + \beta_{r_2} + \beta_{r_3} + \beta_{r_4} + \beta_{c_1} + \beta_{c_2} + \beta_{c_3} + \beta_{c_4}$$

$$\mathcal{Z} = \gamma_{r_1} + \gamma_{r_2} + \gamma_{r_3} + \gamma_{r_4} + \gamma_{c_1} + \gamma_{c_2} + \gamma_{c_3} + \gamma_{c_4}$$

$$\mathcal{T} = \theta_{r_1} + \theta_{r_2} + \theta_{r_3} + \theta_{r_4} + \theta_{c_1} + \theta_{c_2} + \theta_{c_3} + \theta_{c_4}$$

$$\mathcal{X}_{dim} = \alpha_{r_2} + \alpha_{r_3} + \alpha_{r_4} + \alpha_{c_1} + \alpha_{c_2} + \alpha_{c_3} + \alpha_{c_4}$$

$$\mathcal{Y}_{dim} = \beta_{r_2} + \beta_{r_3} + \beta_{r_4} + \beta_{c_1} + \beta_{c_2} + \beta_{c_3} + \beta_{c_4}$$

$$\mathcal{Z}_{dim} = \gamma_{r_2} + \gamma_{r_3} + \gamma_{r_4} + \gamma_{c_1} + \gamma_{c_2} + \gamma_{c_3} + \gamma_{c_4}$$

$$\mathcal{T}_{dim} = \theta_{r_2} + \theta_{r_3} + \theta_{r_4} + \theta_{c_1} + \theta_{c_2} + \theta_{c_3} + \theta_{c_4}$$

3. $\mathcal{C}_{clnt}$ invokes $\pi_{\text{arithmetic-reshare}}$ for the following shares;

$$\alpha_{r_1}, \beta_{r_1}, \gamma_{r_1}, \theta_{r_1}$$

$$\mathcal{X}_{dim}, \mathcal{Y}_{dim}, \mathcal{Z}_{dim}, \mathcal{T}_{dim}$$

$$\mathcal{X}, \mathcal{Y}, \mathcal{Z}, \mathcal{T}$$

4. Parties in the epoch until the $\mathcal{C}_{clnt}$ invokes $\pi_{\text{arithmetic-reshare}}$ and in the last epoch each share is opened to $\mathcal{C}_{clnt}$.

Lemma 6. *Procedure $\pi_{\text{arithmetic-bucket-sum}}$'s transcript is simulatable against honest majority of 4-party.*

Proof Without loss of generality, assume the worst case scenario which is in both committee there exist a corrupted party such that $P_j \in \mathcal{C}_i$ and $P_1 \in \mathcal{C}_{i+1}$. Double encrypted arithmetic shares $\alpha_{r_1}, \beta_{r_1}, \gamma_{r_1}, \theta_{r_1}, \mathcal{X}_{dim}, \mathcal{Y}_{dim}, \mathcal{Z}_{dim}, \mathcal{T}_{dim}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}, \mathcal{T}$ are shared towards the parties in $\mathcal{C}_{i+1}$ by invoking $\pi_{\text{arithmetic-reshare}}$ which are simulatable by Lemma 3. Assume $s_{j,1}$ is the output of $\pi_{\text{arithmetic-reshare}}$ for α_{r_1} where each party in $\mathcal{C}_i$ share with $P_1 \in \mathcal{C}_{i+1}$. Since P_j is the corrupted party and the other three parties in $\mathcal{C}_i$ are honest parties. Furthermore, pairwise random share $s^{k,l}$ is known only by $P_k \in \mathcal{C}_i$ (where $k \neq j$)

and $P_l \in \mathcal{C}_{i+1}$ (where $l \neq 1$) and $s^{k,l}$ is added to $s^{k,1}$, so $s^{k,l}$ appears uniformly random to $\mathcal{A}$ which implies that $s^{k,1}$ appears uniformly random to $\mathcal{A}$. Therefore, even one part of doubly encrypted r_1 appears uniformly random to $\mathcal{A}$. Same argument holds for $\beta_{r_1}, \gamma_{r_1}, \theta_{r_1}, \mathcal{X}_{dim}, \mathcal{Y}_{dim}, \mathcal{Z}_{dim}, \mathcal{T}_{dim}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}, \mathcal{T}$. Thus, $\mathcal{A}$ has no gain of knowledge about $\alpha_{r_1}, \beta_{r_1}, \gamma_{r_1}, \theta_{r_1}, \mathcal{X}_{dim}, \mathcal{Y}_{dim}, \mathcal{Z}_{dim}, \mathcal{T}_{dim}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}$, and $\mathcal{T}$ more than he should. $\square$

5.4 $\Pi_{\text{private-edaBits}}$

In this section, we provide the details of how $\mathcal{C}_{clnt}$ verifies the correctness of r^1 , r_1 , and the consistency between the binary and arithmetic components of the edaBits. Furthermore, we present the protocol $\Pi_{\text{private-edaBits}}$.

The first procedure described in this section is $\pi_{\text{check-server}}$, which is entirely executed by the parties in $\mathcal{C}_{clnt}$. In this procedure, $\mathcal{C}_{clnt}$ examines the values received from the parties in the final epoch, and subsequently the second procedure is $\pi_{\text{check-}\mathcal{C}_{clnt}}$ which evaluates the consistency across all participating parties.

We now explain the reasoning behind each step of the $\pi_{\text{check-server}}$. First, $\mathcal{C}_{clnt}$ receives the sum $r^1 + r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4 = r + c \in \mathbb{Z}_2^k$ from the protocol $\pi_{\text{binary-bucket-sum}}$, and the arithmetic sum $\mathcal{X} + \mathcal{Y} + \mathcal{Z} + \mathcal{T} = r + c \in \mathbb{Z}_{2^k}$ along with $\mathcal{X}_{dim}, \mathcal{Y}_{dim}, \mathcal{Z}_{dim}, \mathcal{T}_{dim}$ from the protocol $\pi_{\text{arithmetic-bucket-sum}}$.

Additionally, the value r_1 is obtained by opening the shares $\alpha_{r_1}, \beta_{r_1}, \gamma_{r_1}, \theta_{r_1}$, and the value r^1 is reconstructed from $\alpha_{r^1}, \beta_{r^1}, \gamma_{r^1}, \theta_{r^1}$. The client $\mathcal{C}_{clnt}$ first checks the consistency between the binary and arithmetic bucket sums, this being the main objective of the protocol, by comparing $r + c \in \mathbb{Z}_{2^k}$ with $r + c \in \mathbb{Z}_2$.

If the comparison is consistent, then $\mathcal{C}_{clnt}$ proceeds to validate the correctness of both the binary and arithmetic components of the private edaBits.

For the arithmetic part of the private edaBits, $\mathcal{C}_{clnt}$ uses the value $r + c \in \mathbb{Z}_{2^k}$, the sum $\mathcal{X}_{dim} + \mathcal{Y}_{dim} + \mathcal{Z}_{dim} + \mathcal{T}_{dim}$, and the value r_1 . The client adds r_1 to the sum $\mathcal{X}_{dim} + \mathcal{Y}_{dim} + \mathcal{Z}_{dim} + \mathcal{T}_{dim}$, and checks whether the result is consistent with $r + c \in \mathbb{Z}_{2^k}$. If the values match, then $\mathcal{C}_{clnt}$ has a valid candidate for the correct arithmetic part of the edaBits, namely r_1 .

For the binary part of the private edaBits, $\mathcal{C}_{clnt}$ uses $r + c \in \mathbb{Z}_2$, the value $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ from $\pi_{\text{binary-bucket-sum}}$, and the value r^1 . The client performs the bit addition of r^1 and $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$, and checks whether the result is consistent with $r + c \in \mathbb{Z}_2$. If the values are consistent, then $\mathcal{C}_{clnt}$ has a valid candidate for the correct binary part of the edaBits, namely r^1 .

You can investigate the details of the protocol is below.

Procedure $\pi_{\text{check-server}}$

Input: $\mathcal{C}_{clnt}$ obtains $r^1 + r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4 = r + c \in \mathbb{Z}_2$, $\mathcal{X} + \mathcal{Y} + \mathcal{Z} + \mathcal{T} = r + c \in \mathbb{Z}_{2^k}$, $\mathcal{X}_{dim}, \mathcal{Y}_{dim}, \mathcal{Z}_{dim}, \mathcal{T}_{dim}$, r_1 obtained by opening $\alpha_{r_1}, \beta_{r_1}, \gamma_{r_1}, \theta_{r_1}$ and r^1 obtained by $\alpha_{r^1}, \beta_{r^1}, \gamma_{r^1}, \theta_{r^1}$.

Usage: $\mathcal{C}_{clnt}$ obtains the secret shares from the parties $\in \mathcal{C}_{i+2}$ and checks the correctness of the secrets.

1. $\mathcal{C}_{clnt}$ takes $r^1 + r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ and deletes the bits after k -th bit. Then invokes $\mathcal{F}_{\text{prep}}$ and converts binary to arithmetic.
2. $\mathcal{C}_{clnt}$ checks the consistency of converted sum and $\mathcal{X} + \mathcal{Y} + \mathcal{Z} + \mathcal{T} = r + c$. If they are not consistent then aborts the protocol.
3. $\mathcal{C}_{clnt}$ takes $r^1 + r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4 = r + c \in \mathbb{Z}_2$, $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ and r^1 deletes the bits after k -th bit. Then makes bit addition on $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ and r^1 and checks the consistency of result of the addition and $r + c \in \mathbb{Z}_2$. If they are not consistent then aborts the protocol.
4. $\mathcal{C}_{clnt}$ takes r_1 and $r + c$ subtract r_1 from $r + c$ then check the consistency of $r_2 + r_3 + r_4 + c_1 + c_2 + c_3 + c_4 \stackrel{?}{=} \mathcal{X}_{dim} + \mathcal{Y}_{dim} + \mathcal{Z}_{dim} + \mathcal{T}_{dim}$. If they are not consistent then aborts the protocol.

Lemma 7. *The transcript of $\pi_{\text{check-server}}$ is simulatable against honest majority of 4-party.*

Proof Each input $\mathcal{C}_{clnt}$ has is the output of previous procedures and they are simulatable by Lemma 1, Lemma 2, Lemma 3, Lemma 4 and Lemma 5. $\pi_{\text{check-server}}$ is related to $\mathcal{C}_{clnt}$ and he executes the procedure locally, so there is no interaction with other parties after he obtains the input. Therefore the only possible scenarios are whether $\mathcal{C}_{clnt}$ or not. Assume the $\mathcal{C}_{clnt}$ is a corrupted party. $\mathcal{C}_{clnt}$ obtains $r + c$ as its binary and arithmetic form. In order to make the gain of information about r he has to find c which has the probability of $\frac{1}{2^k}$ for both domains. He also cannot make the gain of information about r by using the diminished sums because he still has to find c . Therefore $r + c$ and diminished sums seem uniformly random to the adversary $\mathcal{A}$. $\square$

The second verification procedure is denoted as $\pi_{\text{check-}\mathcal{C}_{clnt}}$, which ensures the consistency between the binary and arithmetic components of the edaBits. We begin by presenting the reasoning behind the procedure, followed by a description of its execution.

Even in the absence of an abort during the execution of $\pi_{\text{check-server}}$, inconsistencies between the binary and arithmetic components of the edaBits may still occur. For example, a malicious party $P_j \in \mathcal{C}_{clnt}$ may construct two edaBits such that the binary part of the first edaBit differs by one bit from its corresponding arithmetic value. This inconsistency can be masked by modifying the binary part of a second edaBit in the same bucket. This vulnerability arises due to the use of batched summation: since the protocol aggregates edaBits across the bucket, such adversarial adjustments may remain undetected.

To address this issue, the protocol proceeds as follows. Each party $P_i \in \mathcal{C}_{clnt}$ uniformly samples $\{(b_{i,0}, b_{i,1}, \dots, b_{i,k-1})\}_{i=1}^{24} \in \mathbb{Z}_2^k$ additional edaBits to be used as inputs for $\pi_{\text{binary-bucket-sum}}$, executed within the same server. These edaBits are grouped into 8 per bucket, resulting in 4 buckets per party $P_j \in \mathcal{C}_{clnt}$. The protocol $\pi_{\text{binary-bucket-sum}}$ is executed, and the resulting outputs are distributed to the parties in $\mathcal{C}_{clnt}$. Subsequently, the parties invoke $\pi_{\text{check-server}}$ to verify the correctness of the edaBits.

At the conclusion of this process, each party in $\mathcal{C}_{clnt}$ receives four edaBits from different parties. For example, $P_1 \in \mathcal{C}_{clnt}$ receives edaBits denoted r, s, t , and z , where

each edaBit includes both binary and arithmetic components. Specifically, P_1 obtains r from the first party, p from the second, t from the third, and z from the fourth. Among these, the first edaBit (i.e., r) is treated as the candidate private edaBit, while the remaining three (s , t , z) serve as control edaBits.

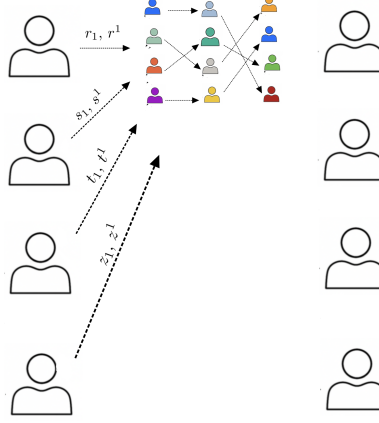


Figure 2: The first party in $\mathcal{C}_{clnt}$ obtains the candidate edaBits and control edaBits

Each party then locally sums the binary and arithmetic parts of their control edaBits. For P_1 , the summations are as $c_1 = s_1 + t_1 + z_1$, $c^1 = s^1 + t^1 + z^1$. Next, they add the components of their candidate edaBit to the control sums $d_1 = r_1 + c_1$, $d^1 = r^1 + c^1$.

Note that each party holds a share of each edaBits (r , s , t , z), so collectively, the parties hold a secret sharing of $r + c$ where P_1 holds d_1 , P_2 holds d_2 , and so on. The parties then open their shares and obtain values $d \in \mathbb{Z}_{2^k}$ and $d \in \mathbb{Z}_2^k$. If these two representations are consistent, then r is accepted as a valid private edaBit. Otherwise, the protocol is aborted.

The security of $\pi_{\text{check-}\mathcal{C}_{clnt}}$ derives from the design of the construction. Since there is at most one corrupted party in $\mathcal{C}_{clnt}$, the honest parties will produce correct and consistent edaBits. If the corrupted party attempts to introduce inconsistencies, the discrepancy between the binary and arithmetic components will be revealed when the shares are opened, thus ensuring the detection of cheating behavior.

Procedure $\pi_{\text{check-}\mathcal{C}_{clnt}}$

Input: Each party P_j has r_j, r^j , s_j, s^j , t_j, t^j , z_j, z^j as an output of $\pi_{\text{binary-bucket-sum}}$ generated by different parties.

Usage: Parties in $\mathcal{C}_{clnt}$ checks the consistency of binary and arithmetic form of the bucket sum.

1. Each party P_j locally execute the following control sum

$$c_j = s_j + t_j + z_j, \quad c^j = s^j + t^j + z^j.$$

2. Parties add the components of their candidate edaBit to the control sums:

$$d_j = r_j + c_j, \quad d^j = r^j + c^j.$$

3. Parties open the shares and obtain $d \in \mathbb{Z}_{2^k}$ and $d \in \mathbb{Z}_2^k$
4. Parties check the consistency between $d \in \mathbb{Z}_{2^k}$ and $d \in \mathbb{Z}_2^k$. If they are not consistent, then abort the protocol.

Lemma 8. *The transcript of $\pi_{\text{check-}\mathcal{C}_{\text{clnt}}}$ is simulatable against honest majority of 4-party.*

Proof Each input available to the parties in $\mathcal{C}_{\text{clnt}}$ is derived from the outputs of the preceding procedures and is simulatable, as established in Lemma 1, Lemma 2, Lemma 3, Lemma 4, and Lemma 5. The procedure $\pi_{\text{check-}\mathcal{C}_{\text{clnt}}}$ specifically is related to the parties in $\mathcal{C}_{\text{clnt}}$. Without loss of generality, we consider the worst-case scenario in which there exists one corrupted party in each epoch. This setup represents the case in which the adversary $\mathcal{A}$ can potentially gain the most information. Since there are three honest parties in $\mathcal{C}_{\text{clnt}}$, and each generates correct and consistent edaBits, the aggregated sum of their edaBits also remains correct and consistent. If the edaBit contributed by the corrupted party is inconsistent, this inconsistency is revealed upon opening the summation, due to the integrity of the three honest contributions. In summary, each party possesses three reliable and one potentially corrupted edaBit. When these edaBits are summed to compute c and d , the results appear uniformly random from the adversary $\mathcal{A}$, thus ensuring that no meaningful information is leaked. $\square$

Now we arrive the private edaBits protocol, $\Pi_{\text{private-edaBits}}$. The protocol consist of the procedures which we explain until now but small differences.

Protocol $\Pi_{\text{private-edaBits}}$

Input: $\mathcal{C}_{\text{clnt}}$ samples bits $\{([b_{j,0}]_2, [b_{j,1}]_2, \dots, [b_{j,m-1}]_2)\}_{j=1}^8$

Output: $\mathcal{C}_{\text{clnt}}$ has one private edaBits $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$

1. $\mathcal{C}_{\text{clnt}}$ invokes $\mathcal{F}_{\text{prep}}$ to convert the bits from binary to arithmetic as $b^i = r_i$ for $i = 1, 2, 3, 4$ and $b^i = c_i$ for $i = 5, 6, 7, 8$. Then sets $r = r_1 + r_2 + r_3 + r_4 \in \mathbb{Z}_{2^k}$ and $c = c_1 + c_2 + c_3 + c_4 \in \mathbb{Z}_{2^k}$.
2. $\mathcal{C}_{\text{clnt}}$ invokes $\pi_{\text{arithmetic-double-encryption}}$ and $\pi_{\text{binary-double-encryption}}$ on his binary shares and arithmetic shares.
3. $\mathcal{C}_{\text{clnt}}$ invokes $\pi_{\text{arithmetic-bucket-sum}}$ and gets $\alpha_{r_1}, \beta_{r_1}, \gamma_{r_1}, \theta_{r_1}, \mathcal{X}_{\text{dim}}, \mathcal{Y}_{\text{dim}}, \mathcal{Z}_{\text{dim}}, \mathcal{T}_{\text{dim}}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}, \mathcal{T}$.
4. $\mathcal{C}_{\text{clnt}}$ and all parties in each epochs $\mathcal{C}^1, \mathcal{C}^2, \mathcal{C}^3$ invokes $\pi_{\text{arithmetic-reshare}}$ on the arithmetic secret shares.

5. In parallel, $\mathcal{C}_{clnt}$ and all parties in each epochs $\mathcal{C}^1, \mathcal{C}^2, \mathcal{C}^3$ invokes $\pi_{\text{binary-reshare}}$ on the binary secret shares and invokes $\pi_{\text{binary-bucket-sum}}$.
6. If there is no abort until epoch $\mathcal{C}^3$ then the parties open their secrets to $\mathcal{C}_{clnt}$.
7. $\mathcal{C}_{clnt}$ invokes $\pi_{\text{check-server}}$ and he checks consistency of r_1 and r^1 obtained from other rounds and if correctness of r_1 and r^1 fails, abort the protocol.
8. $\mathcal{C}_{clnt}$ invokes $\pi_{\text{check-}\mathcal{C}_{clnt}}$ and he checks consistency of r_1 and r^1 obtained from other rounds and if r_1 and r^1 is not consistent, abort the protocol.
9. If $\mathcal{C}_{clnt}$ does not abort the protocol in previous steps then he obtains $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$.

Theorem 2. *Let $\mathcal{A}$ be an adaptive adversary in $\Pi_{\text{private-edaBits}}$. Then the protocol UC-securely realizes the $\mathcal{F}_{\text{priv-edaBits}}$ against honest majority of 4-party in the presence of $\mathcal{A}$ in the $\mathcal{F}_{\text{prep}}, \mathcal{F}_{\text{ABB}}, \mathcal{F}_{\text{next-com-priv}}$ - hybrid model.*

Proof We begin with constructing a simulator $\mathcal{S}$ and an adversary $\mathcal{A}$ which is the subroutine of $\mathcal{S}$. The adversary is given access to $\mathcal{F}_{\text{prep}}, \mathcal{F}_{\text{ABB}}$ and $\mathcal{F}_{\text{next-com-priv}}$ functionalities. We assume that all communication is between the environment $\mathcal{Z}$ and $\mathcal{A}$. The simulator is initially is set **status** = 1 means there is no abort. If the simulator observe any bad behaviour then it sets **status** = 0. The simulation is as follows;

Init: On input (**Init**, $\mathcal{C}$) we rely on $\mathcal{F}_{\text{prep}}$ about being leakage free of the step. (For this round it may not be understandable the behavior of $\mathcal{A}$ and inputs sent to next committees by $\mathcal{A}$, we eventually can overcome this with other steps, see below)

Input On input (**Input**, id_x) simulate $\pi_{\text{binary-reshare}}$ as in Lemma 1, $\pi_{\text{binary-double-encryption}}$ as in Lemma 2, $\pi_{\text{arithmetic-reshare}}$ as in Lemma 3 and $\pi_{\text{arithmetic-double-encryption}}$ as in Lemma 4. We are going to use this step in the next steps in order to realize the simulation well.

Binary Bucket Sum: On input (**BBS**, $\text{id}_x, \text{id}_y, \text{id}_z$) the simulator simulates $\pi_{\text{binary-reshare}}$ as in Lemma 1 and $\pi_{\text{binary-double-encryption}}$ as in Lemma 2. $\mathcal{S}$ simulates the honest parties' binary parts of edaBits obtained by previous committee and receives the bit triples from $\mathcal{A}$. Observe that $\mathcal{S}$ has the all 4 bit triples. If $\mathcal{A}$ cheats by sending incorrect binary triples to $\mathcal{S}$ and the simulator can understand immediately because $\mathcal{S}$ has all bit triples. Then $\mathcal{S}$ updates the **status**= 0 and abort the protocol because observe that there is only one possible corrupted party candidate thanks to our honest majority of 4-party construction.

Arithmetic Bucket Sum: On input (**ABS**, $\text{id}_x, \text{id}_y, \text{id}_z$) the simulator simulates $\pi_{\text{arithmetic-reshare}}$ as in Lemma 3 and $\pi_{\text{arithmetic-double-encryption}}$ as in Lemma 4. We are going to use this step in the next step in order to realize the simulation well.

Check: On input (**check_{server}**, id_z) simulator takes the inputs from $\mathcal{A}$. Since the simulator opens the binary bucket sum perfectly secure it can detect any incorrect data from $\mathcal{A}$. We divide into two parts this section. For the first part, we investigate binary part. $\mathcal{A}$ cannot cheat during the binary bucket sum step because $\mathcal{S}$ obtains the binary bucket sum with perfect security. Then binary bucket sum, $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ and r^1 . is opened to $\mathcal{S}$ and $\mathcal{S}$ can check the correctness of the binary part of the edaBits by making the bit addition of $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ and r^1 . If $\mathcal{A}$ cheats during resharing the binary parts then the binary bucket sum and sum of $r^2 + r^3 + r^4 + c^1 + c^2 + c^3 + c^4$ and r^1 becomes inconsistent. Then, $\mathcal{S}$ updates the **status**= 0. For the arithmetic part, $\mathcal{S}$ uses the security of the binary bucket sum again. $\mathcal{S}$ convert the binary bucket sum to arithmetic and check the consistency between the opened values. If they are not consistent $\mathcal{S}$ updates the **status**= 0. If **status**

$= 1$, $\mathcal{S}$ continue to check the correctness of arithmetic part of the edaBits by adding r_1 to $\mathcal{X}_{dim} + \mathcal{Y}_{dim} + \mathcal{Z}_{dim} + \mathcal{T}_{dim}$. If the sum does not equal $r + c \in \mathbb{Z}_{2^k}$, then $\mathcal{S}$ updates **status** = 0. For the second part of the check, on input $(\text{check}_{\text{client}}, \text{id}_z)$ simulator takes the input from $\mathcal{A}$. Since the candidate private edaBits' and control edaBits' correctness is guaranteed by $\pi_{\text{binary-bucket-sum}}$ and edaBits does not leak any information by Lemma 3 and Lemma 6. Since $\mathcal{S}$ have all edaBits from each party, s and c can be opened without any leakage. Then $\mathcal{S}$ check the consistency between $s \in \mathbb{Z}_{2^k}$ and $s \in \mathbb{Z}_2^k$. If they are not consistent, $\mathcal{S}$ updates **status** = 0.

If $\mathcal{S}$ updates the **status** = 0 then aborts the protocol and send to $\mathcal{A}$ random values. Therefore, our functionality is simulatable securely. $\square$

5.5 Complexity Analysis of $\Pi_{\text{private-edaBits}}$

In this chapter, we analyze the memory, communication, and time complexity of the procedures individually. After evaluating these costs, we present the overall complexity of the protocol $\Pi_{\text{private-edaBits}}$. We begin with the memory requirements of the procedures from the perspective of a single party for one epoch of computation, and subsequently compute the communication and time complexity of the parties.

We start with the memory complexity. In $\pi_{\text{arithmetic-reshare}}$, a party is required to store 4 arithmetic ring elements as pairwise random secret shares. For $\pi_{\text{arithmetic-double-encryption}}$, the requirement increases to 16 arithmetic ring elements. The final procedure in the arithmetic part of edaBits, $\pi_{\text{arithmetic-bucket-sum}}$, is a combination of $\pi_{\text{arithmetic-reshare}}$ and $\pi_{\text{arithmetic-double-encryption}}$, and thus its cost depends on these two procedures. Specifically, in $\pi_{\text{arithmetic-bucket-sum}}$, a party needs to double encrypt two arithmetic numbers, which requires 32 arithmetic ring elements. Additionally, 48 ring elements are necessary for sending these arithmetic secrets to the next committee. Hence, in total, the arithmetic part of $\Pi_{\text{private-edaBits}}$ requires 80 arithmetic ring elements per party.

For the binary part of $\Pi_{\text{private-edaBits}}$, a party must hold $4k$ bits for $\pi_{\text{binary-reshare}}$, where k denotes the length of the binary component of edaBits. Subsequently, $32k$ bits are needed for $\pi_{\text{binary-double-encryption}}$ to handle the binary forms of r and c . To transmit the double-encrypted bits and the binary part of the candidate edaBits, $20k$ bits are required. Furthermore, depending on the committee membership, a party must additionally store $2k$ bits (for the first and third committees) or $3k$ bits (for the second committee) during $\pi_{\text{binary-bucket-sum}}$. Consequently, the storage requirements for a party amount to $54k$ bits if they belong to the first or third committee, and $55k$ bits if they belong to the second committee. Thus, the binary part of $\Pi_{\text{private-edaBits}}$ requires at least $87k$ bits. Combining both parts, one party needs $80 + 87k$ arithmetic ring elements and bits. Since $\Pi_{\text{private-edaBits}}$ is executed in parallel 4 times, the total requirement per party becomes $320 + 348k$. Therefore, the memory complexity of $\Pi_{\text{private-edaBits}}$ is $\mathcal{O}(k)$.

We next examine the time complexity of $\Pi_{\text{private-edaBits}}$. The protocol involves arithmetic additions and k -bit additions. Since the time complexity of arithmetic addition is $\mathcal{O}(1)$, the main cost arises from bit additions. In the binary part of $\Pi_{\text{private-edaBits}}$, each party must perform 15 instances of k -bit addition. As a result, the time complexity of $\Pi_{\text{private-edaBits}}$ is $\mathcal{O}(k)$.

Finally, we evaluate the communication complexity of $\Pi_{\text{private-edaBits}}$. In previous MPC protocols designed within the fluid MPC framework [15, 16], the communication complexity is $\mathcal{O}(n^2)$ and $\mathcal{O}(n)$ respectively, where n denotes the number of parties. Although in each epoch parties exchange information with the next committees in $\mathcal{O}(n^2)$, in our setting the number of parties is fixed at $n = 4$. Since n is constant, the communication complexity of $\Pi_{\text{private-edaBits}}$ simplifies to $\mathcal{O}(16) = \mathcal{O}(1)$. Thus, the communication complexity of the protocol is constant.

In conclusion, $\Pi_{\text{private-edaBits}}$ achieves $\mathcal{O}(k)$ memory complexity, $\mathcal{O}(k)$ time complexity, and $\mathcal{O}(1)$ communication complexity, benefiting from the fixed 4-party structure of the design.

6 Applications

In this section, we present two applications of edaBits in the MPC setting against semi-honest adversaries. The first is the arithmetic-to-binary conversion protocol, denoted by Π_{A2B} , and the second is the binary-to-arithmetic conversion protocol, denoted by Π_{B2A} .

In various applications, such as privacy-preserving machine learning, converting secret shares between arithmetic and binary representations is essential. This conversion becomes particularly important when optimizing algorithmic efficiency, as some computations are more efficient in the binary domain, while others perform better in the arithmetic domain.

6.1 From Arithmetic to Binary

The first application, Π_{A2B} , converts arithmetic shares into binary shares. In this setting, the parties are provided with edaBits during the preprocessing phase and use them to obtain the binary representation of their arithmetic share as follows.

Each party begins with an arithmetic secret $[x]$ and the arithmetic component of edaBits $[r]$ obtained via Π_{edaBits} . They compute $[c] \leftarrow [x] - [r]$ in the arithmetic domain. To recover $[x]$, one would typically add $[r]$ to $[c]$. However, in this case, the parties perform this addition using the binary representation of the edaBits. Thus, they effectively convert $[x]$ into its binary form. The detailed steps of this conversion protocol are described below.

Protocol Π_{A2B}

Usage: Parties in $\mathcal{C}_i$ have the arithmetic secret share $[x]_M$ and parties in $\mathcal{C}_{i+1}$ obtains binary secret shares $([x_0]_2, \dots, [x_{k-1}]_2)$.

1. Parties in $\mathcal{C}_i$ invoke $\mathcal{F}_{\text{edaBitsFluid}}$ to have $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$.
2. Parties locally $[c]_M \leftarrow [x]_M - [r]_M$.
3. Parties open c to parties $\in \mathcal{C}_{i+1}$. In parallel, they invoke $\pi_{\text{binary-reshare}}$ on each bit of the $[b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2$.
4. Parties in $\mathcal{C}_{i+1}$ make the bit addition as follows;

$$([x_0]_2, [x_1]_2, \dots, [x_{k-1}]_2) \leftarrow \text{BitAdd}(c, [b_0]_2, \dots, [b_{k-1}]_2).$$

The reason for choosing a semi-honest adversary model lies in the handling of the binary part of the edaBits. Specifically, the procedure $\pi_{\text{binary-reshare}}$ is needed to have a control mechanism, such as the control bit triples employed in $\pi_{\text{binary-bucket-sum}}$, which could otherwise detect or prevent manipulation. Consequently, transferring binary shares without such a mechanism may lead to potential leakage.

However, under the semi-honest adversary model, corrupted parties are assumed to follow the protocol honestly and do not manipulate with the secret binary shares output by $\pi_{\text{binary-reshare}}$. Furthermore, according to Lemma 1, the procedure $\pi_{\text{binary-reshare}}$ does not cause any leakage in this setting.

Theorem 3. *The transcript of Π_{A2B} is simulatable against semi-honest adversary.*

Proof We know that $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$ seem to be uniformly random to $\mathcal{A}$ by Theorem 2, so subtracting r from secret share x and obtaining c seems still uniformly random. Therefore, opening c is simulatable by random values. Furthermore, we know that the procedure $\pi_{\text{binary-reshare}}$ is simulatable by random values from Lemma 1.

$\mathcal{A}$ cannot make of gain any information about the secret share x and private edaBits as we explained above, also since he is semi-honest he follows the protocol which means that there is not any manipulation on edaBits. Therefore, we can guarantee that the protocol is complete. As a conclusion Π_{A2B} is simulatable. $\square$

6.2 From Binary to Arithmetic

The protocol Π_{B2A} is so similar with Π_{A2B} , but reverse order.

Protocol Π_{B2A}

Usage: Parties in $\mathcal{C}_i$ have the binary secret shares $([x_0]_2, \dots, [x_{k-1}]_2)$ and parties in $\mathcal{C}_{i+1}$ obtains arithmetic secret share $[x]_M$

1. Parties in $\mathcal{C}_i$ call $\mathcal{F}_{\text{edaBitsFluid}}$ to have $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$.
2. Parties locally $([c_0]_2, [c_1]_2, \dots, [c_{k-1}]_2) \leftarrow \text{BitAdd}([x_0]_2, [x_1]_2, \dots, [x_{k-1}]_2, ([b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2))$.
3. Parties open $[c_i]_2$ for $i = 0, \dots, k-1$ to parties $\in \mathcal{C}_{i+1}$. In parallel, they invoke $\pi_{\text{arithmetic-reshare}}$ on $[r]_M$.
4. Parties in $\mathcal{C}_{i+1}$ make the bit subtraction as follows;

$$c = \sum_{i=0}^{k-1} c_i \cdot 2^i, \text{ then they output } [x]_M \leftarrow c - [r]_M$$

The differences between the protocols lie in the second and the final steps. In the second step, the parties possess binary secret shares and perform bit addition between their secret binary share and the binary part of their edaBits. In the final step, they

subtract the resulting value $[c]$ from the arithmetic part of their edaBits in order to obtain the original arithmetic share $[x]$.

Theorem 4. *The transcript of Π_{B2A} is simulatable against semi-honest adversary.*

Proof We know that the procedure $\pi_{\text{binary-reshare}}$ is simulatable by random values from Lemma 1. Furthermore, we know that $([r]_M, [b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2)$ seem to be uniformly random to $\mathcal{A}$ by Theorem 2, so making bit addition of binary form of the secret share of x and binary form of the edaBits $[b_0]_2, [b_1]_2, \dots, [b_{k-1}]_2$ and obtaining c seems still uniformly random. Therefore, opening c is simulatable by random values.

As we explained above, the adversary cannot gain any information about the secret share x and private edaBits, also since he is semi-honest, we can guarantee that the protocol is complete. Therefore, Π_{B2A} is simulatable. $\square$

6.3 Complexity Analysis of Applications

In this chapter, we evaluate the memory, communication, and time complexity of the protocols Π_{A2B} and Π_{B2A} . Owing to their close structural similarity, we analyze their costs jointly.

Memory complexity. In both protocols, a party invokes $\mathcal{F}_{\text{edaBitsFluid}}$ to obtain an edaBit, consisting of one arithmetic ring element and a k -bit binary value. In Π_{A2B} , each party initially holds an arithmetic secret share and must transmit the binary part of the edaBits to the next committee by executing $\pi_{\text{binary-reshare}}$. This requires storing $4k$ bits as pairwise binary secret shares. Thus, the overall storage requirement per party is $2 + 5k$ (arithmetic ring elements and bits). In contrast, in Π_{B2A} , parties hold binary secret shares of length k and must transmit the arithmetic part of the edaBits using $\pi_{\text{arithmetic-reshare}}$. Accordingly, each party must store $5 + 2k$ arithmetic ring elements and bits. Hence, the memory complexity of both Π_{A2B} and Π_{B2A} is $\mathcal{O}(k)$.

Time complexity. The computation in both protocols is limited to k -bit additions. Therefore, the time complexity of Π_{A2B} and Π_{B2A} is $\mathcal{O}(k)$.

Communication complexity. During execution, parties must communicate with every party in the next committee, yielding $\mathcal{O}(n^2)$ communication complexity. Since our design fixes $n = 4$, this simplifies to $\mathcal{O}(16)$, a constant bound. Consequently, the communication complexity of Π_{A2B} and Π_{B2A} is $\mathcal{O}(1)$.

To conclude, we presented the Arithmetic-to-Binary (A2B) and Binary-to-Arithmetic (B2A) protocols, both secure against semi-honest adversaries within the fluid MPC framework. Their security was established in the UC model, and we analyzed their memory, time, and communication complexity.

7 Conclusion

In this work, we introduced *edaBits*, a preprocessing protocol for secure multi-party computation (MPC), and studied its integration into the fluid MPC and UC frameworks. The motivation stems from the limitations of conventional MPC protocols, where continuous party participation and inefficiency hinder practical deployment.

The edaBits protocol addresses these issues by accelerating computations, especially in privacy-preserving machine learning.

We embedded edaBits into the fluid MPC framework, introducing seven novel procedures under a four-party honest-majority model. This design ensures correctness with negligible probability of misbehavior. Additionally, we demonstrated two applications where edaBits enhance computations against semi-honest adversaries.

Our complexity analysis shows that $\Pi_{\text{private-edaBits}}$, along with applications A2B and B2A, achieves linear memory and time complexity $\mathcal{O}(k)$, while attaining constant communication complexity through the four-party design.

In conclusion, we proposed a new edaBits protocol for the fluid MPC setting with efficient complexity guarantees and practical applications. This work lays the foundation for future extensions, including protocols for the n -party dishonest-majority setting and applications resilient to malicious adversaries.

References

- [1] Yao, A.C.: Protocols for secure computations. In: 23rd Annual Symposium on Foundations of Computer Science (sfcs 1982), pp. 160–164 (1982). IEEE
- [2] Wigderson, A., Or, M., Goldwasser, S.: Completeness theorems for noncryptographic fault-tolerant distributed computations. In: Proceedings of the 20th Annual Symposium on the Theory of Computing (STOC’88), pp. 1–10 (1988)
- [3] Micali, S., Goldreich, O., Wigderson, A.: How to play any mental game. In: Proceedings of the Nineteenth ACM Symp. on Theory of Computing, STOC, pp. 218–229 (1987). ACM New York
- [4] Rabin, T., Ben-Or, M.: Verifiable secret sharing and multiparty protocols with honest majority. In: Proceedings of the Twenty-first Annual ACM Symposium on Theory of Computing, pp. 73–85 (1989)
- [5] Shamir, A.: How to share a secret. Communications of the ACM **22**(11), 612–613 (1979)
- [6] Yao, A.C.-C.: How to generate and exchange secrets. In: 27th Annual Symposium on Foundations of Computer Science (Sfcs 1986), pp. 162–167 (1986). IEEE
- [7] Demmler, D., Schneider, T., Zohner, M.: Aby-a framework for efficient mixed-protocol secure two-party computation. In: NDSS (2015)
- [8] Escudero, D., Ghosh, S., Keller, M., Rachuri, R., Scholl, P.: Improved primitives for MPC over mixed arithmetic-binary circuits. In: Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part II 40, pp. 823–852 (2020). Springer
- [9] Garg, R., Yang, K., Katz, J., Wang, X.: Scalable mixed-mode mpc. In: 2024 IEEE

Symposium on Security and Privacy (SP), pp. 523–541 (2024). IEEE

- [10] Mohassel, P., Rindal, P.: Aby3: A mixed protocol framework for machine learning. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 35–52 (2018)
- [11] Rotaru, D., Wood, T.: Marbled circuits: Mixing arithmetic and boolean circuits with active security. In: International Conference on Cryptology in India, pp. 227–249 (2019). Springer
- [12] Canetti, R.: Security and composition of multiparty cryptographic protocols. *Journal of CRYPTOLOGY* **13**, 143–202 (2000)
- [13] Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: Proceedings 42nd IEEE Symposium on Foundations of Computer Science, pp. 136–145 (2001). IEEE
- [14] Canetti, R., Cohen, A., Lindell, Y.: A simpler variant of universally composable security for standard multiparty computation. In: Advances in Cryptology–CRYPTO 2015: 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16–20, 2015, Proceedings, Part II 35, pp. 3–22 (2015). Springer
- [15] Choudhuri, A.R., Goel, A., Green, M., Jain, A., Kaptchuk, G.: Fluid mpc: secure multiparty computation with dynamic participants. In: Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II 41, pp. 94–123 (2021). Springer
- [16] Bienstock, A., Escudero, D., Polychroniadou, A.: On linear communication complexity for (maximally) fluid mpc. In: Annual International Cryptology Conference, pp. 263–294 (2023). Springer
- [17] David, B., Deligios, G., Goel, A., Ishai, Y., Konring, A., Kushilevitz, E., Liu-Zhang, C.-D., Narayanan, V.: Perfect mpc over layered graphs. In: Annual International Cryptology Conference, pp. 360–392 (2023). Springer
- [18] Deligios, G., Goel, A., Liu-Zhang, C.-D.: Maximally-fluid mpc with guaranteed output delivery. *Cryptology ePrint Archive* (2023)
- [19] Gentry, C., Halevi, S., Krawczyk, H., Magri, B., Nielsen, J.B., Rabin, T., Yacoubov, S.: Yoso: You only speak once: Secure mpc with stateless ephemeral roles. In: Annual International Cryptology Conference, pp. 64–93 (2021). Springer
- [20] Rachuri, R., Scholl, P.: Le mans: Dynamic and fluid mpc for dishonest majority. In: Annual International Cryptology Conference, pp. 719–749 (2022). Springer
- [21] Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from

- somewhat homomorphic encryption. In: Annual Cryptology Conference, pp. 643–662 (2012). Springer
- [22] Bendlin, R., Damgård, I., Orlandi, C., Zakarias, S.: Semi-homomorphic encryption and multiparty computation. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 169–188 (2011). Springer
 - [23] Cramer, R., Damgård, I., Escudero, D., Scholl, P., Xing, C.: Spdz2: Efficient mpc mod 2 for dishonest majority. In: Annual International Cryptology Conference. Springer, pp. 769–798 (2018)
 - [24] Frederiksen, T.K., Keller, M., Orsini, E., Scholl, P.: A unified approach to mpc with preprocessing using OT. In: International Conference on the Theory and Application of Cryptology and Information Security, pp. 711–735 (2015). Springer
 - [25] Furukawa, J., Lindell, Y., Nof, A., Weinstein, O.: High-throughput secure three-party computation for malicious adversaries and an honest majority. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 225–255 (2017). Springer